Computer Architecture

Lecture 20b: Multiprocessors

Prof. Onur Mutlu

ETH Zürich

Fall 2025

28 November 2025

Readings: Multiprocessing

- Required
  - Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS 1967.

- Recommended
  - Mike Flynn, "Very High-Speed Computing Systems," Proc. of IEEE, 1966
  - Hill, Jouppi, Sohi, "Multiprocessors and Multicomputers," pp. 551-560 in Readings in Computer Architecture.
  - Hill, Jouppi, Sohi, "Dataflow and Multithreading," pp. 309-314 in Readings in Computer Architecture.

Readings: Memory Consistency

- Required
  - Lamport, "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs," IEEE Transactions on Computers, 1979

Readings: Cache Coherence

- Required
  - Papamarcos and Patel, "A low-overhead coherence solution for multiprocessors with private cache memories," ISCA 1984.

- Recommended:
  - Culler and Singh, *Parallel Computer Architecture*
    - Chapter 5.1 (pp 269 – 283), Chapter 5.3 (pp 291 – 305)
  - P&H, *Computer Organization and Design*
    - Chapter 5.8 (pp 534 – 538 in 4th and 4th revised eds.)

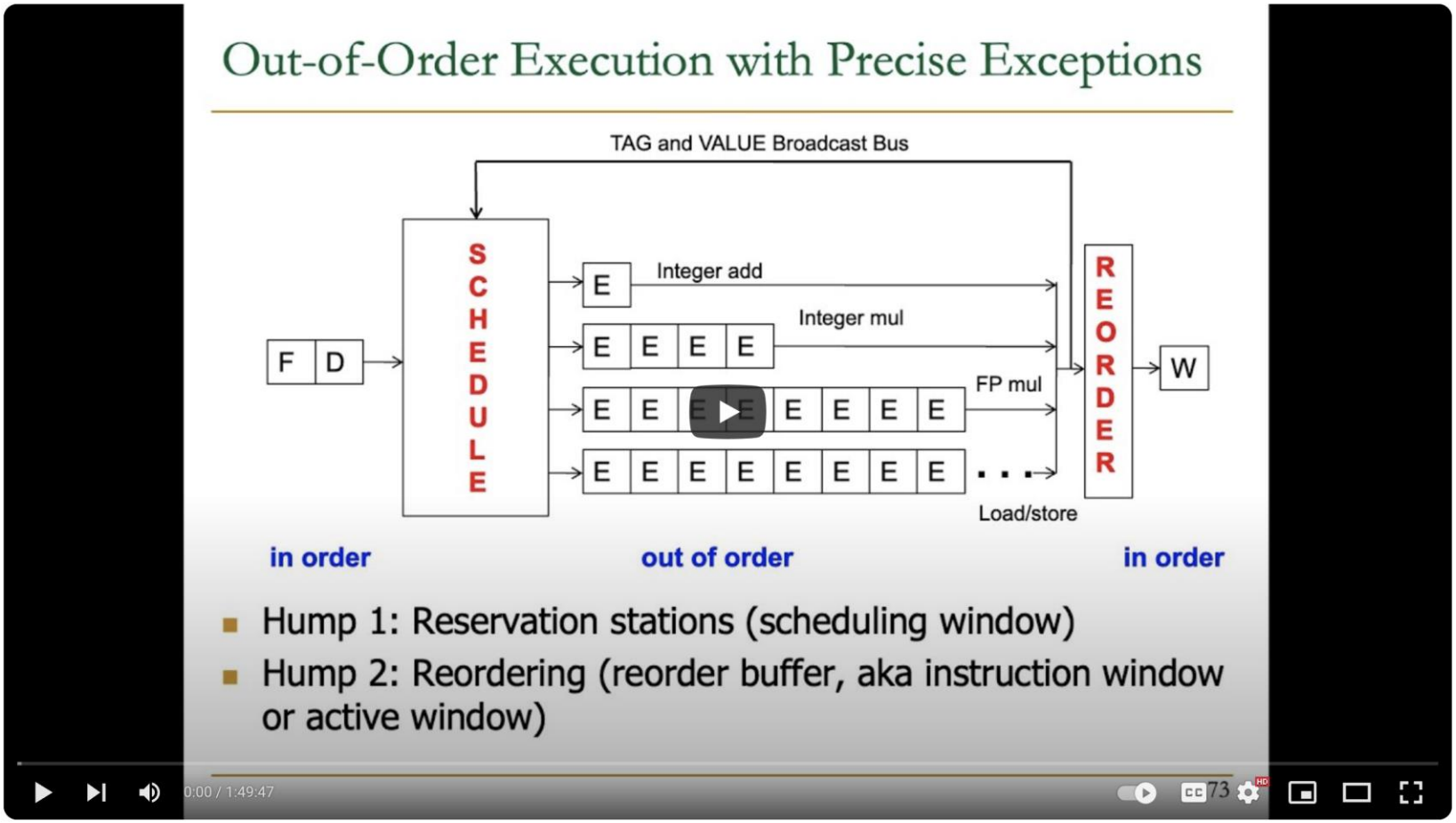
Multiprocessors and Issues in Multiprocessing

Flynn's Taxonomy of Computers

- Mike Flynn, "Very High-Speed Computing Systems," Proc. of IEEE, 1966

- SISD: Single instruction operates on single data element
- SIMD: Single instruction operates on multiple data elements
  - Array processor
  - Vector processor
- MISD: Multiple instructions operate on single data element
  - Closest form: systolic array processor, streaming processor
- MIMD: Multiple instructions operate on multiple data elements (multiple instruction streams)
  - Multiprocessor
  - Multithreaded processor

SISD Example: Scalar, Pipelined, OoO



Digital Design and Computer Architecture - Lecture 15: Out-of-Order Execution (Spring 2023)

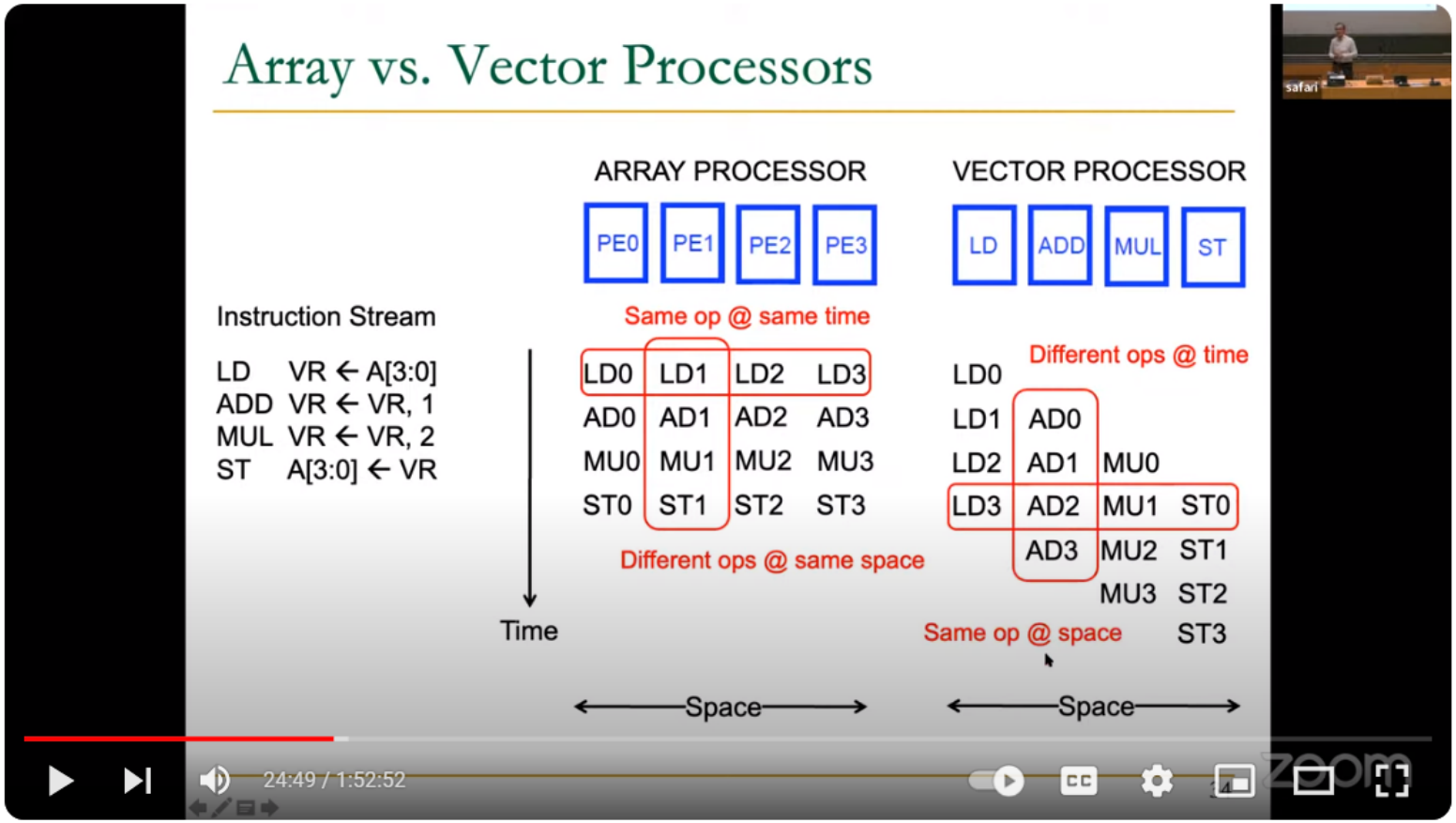
Onur Mutlu Lectures
46.6K subscribers

Subscribed | 154 | Share | Download | Clip | Save

12,154 views Streamed live on Apr 21, 2023
Digital Design and Computer Architecture, ETH Zürich, Spring 2023 https://safari.ethz.ch/digitaltechnik...

https://www.youtube.com/watch?v=AhtlrJNSQws

SIMD Example: Vector & Array Processors



Digital Design and Comp. Arch. - Lecture 19: SIMD Architectures (Vector and Array Processors)

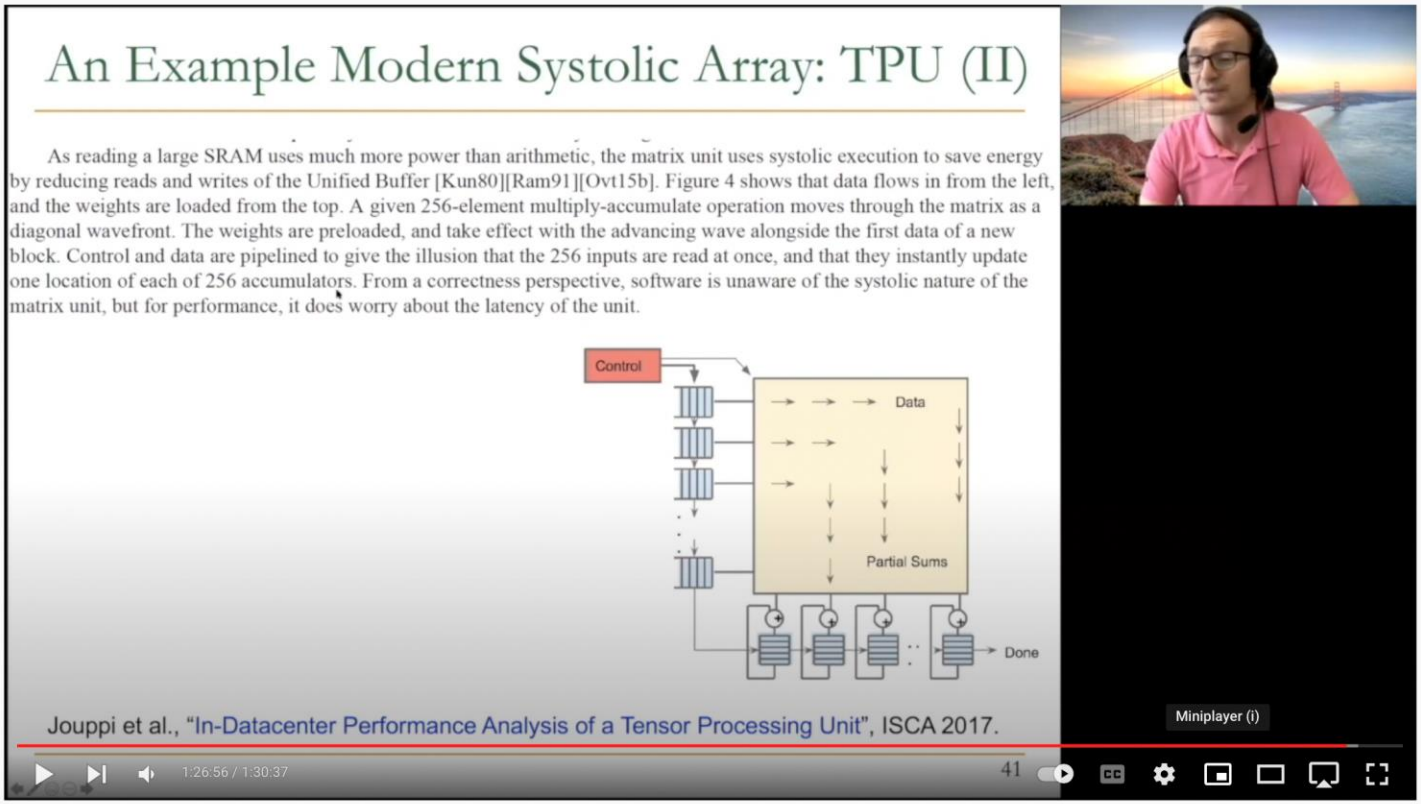
Onur Mutlu Lectures
37.6K subscribers

Subscribed | 72 | Share | Clip

3.1K views Streamed 6 months ago Livestream - Digital Design and Computer Architecture - ETH Zürich (Spring 2023)
Digital Design and Computer Architecture, ETH Zürich, Spring 2023 https://safari.ethz.ch/digitaltechnik...

https://www.youtube.com/watch?v=gkMaO3yJMz0

MISD Example: Systolic Arrays



Digital Design & Computer Arch. - Lecture 19: VLIW and Systolic Array Architectures (Spring 2022)

842 views • Premiered May 6, 2022

35 DISLIKE SHARE CLIP SAVE ...

Onur Mutlu Lectures
24.5K subscribers

SUBSCRIBED

Digital Design and Computer Architecture, ETH Zürich, Spring 2022 (
https://safari.ethz.ch/digitaltechnik...)

Lecture 19a: VLIW Architectures
Lecture 19b: Systolic Array Architectures
Lecturer: Professor Onur Mutlu (https://people.inf.ethz.ch/omutlu/)
Date: May 6, 2022

https://youtu.be/1SSqV7Y75oU?t=2316

Why Parallel Computers?

- Parallelism: Doing multiple things at a time
- Things: instructions, operations, tasks

- Main (or Original) Goal
  - Improve performance (Execution time or task throughput)
    - Execution time of a program governed by Amdahl's Law

- Other Goals
  - Reduce power consumption
    - (4N units at freq F/4) consume less power than (N units at freq F)
    - Why?
  - Improve cost efficiency and scalability, reduce complexity
    - Harder to design a single unit that performs as well as N simpler units
  - Improve dependability: Redundant execution in space

Types of Parallelism and How to Exploit Them

- Instruction Level Parallelism
  - Different instructions within a stream can be executed in parallel
  - Pipelining, out-of-order execution, speculative execution, VLIW
  - Dataflow

- Data Parallelism
  - Different pieces of data can be operated on in parallel
  - SIMD: Vector processing, array processing
  - Systolic arrays, streaming processors

- Task Level Parallelism
  - Different "tasks/threads" can be executed in parallel
  - Multithreading
  - Multiprocessing (multi-core)

Task-Level Parallelism: Creating Tasks

- Partition a single problem into multiple related tasks (threads)
  - Explicitly: Parallel programming
    - Easy when tasks are natural in the problem
      - Web/database queries
    - Difficult when natural task boundaries are unclear
  - Transparently/implicitly: Thread level speculation
    - Partition a single thread speculatively
- Run many independent tasks (processes) together
  - Easy when there are many processes
    - Batch simulations, different users, cloud computing workloads
  - Does not improve the performance of a single task

Multiprocessing Fundamentals

Multiprocessor Types

- Loosely coupled multiprocessors
  - No shared global memory address space
  - Multicomputer network
    - Network-based multiprocessors
  - Usually programmed via message passing
    - Explicit calls (send, receive) for communication

- Tightly coupled multiprocessors
  - Shared global memory address space
  - Traditional multiprocessing: symmetric multiprocessing (SMP)
    - Existing multi-core processors, multithreaded processors
  - Programming model similar to uniprocessors (i.e., multitasking uniprocessor) except
    - Operations on shared data require synchronization

Main Design Issues in Tightly-Coupled MP

- Shared memory synchronization
  - How to handle synchronization: locks, atomic operations, barriers
- Cache coherence
  - How to ensure correct operation in the presence of private caches keeping the same memory address cached
- Memory consistency: Ordering of all memory operations
  - What should the programmer expect the hardware to provide?
- Shared resource management
- Communication: Interconnects

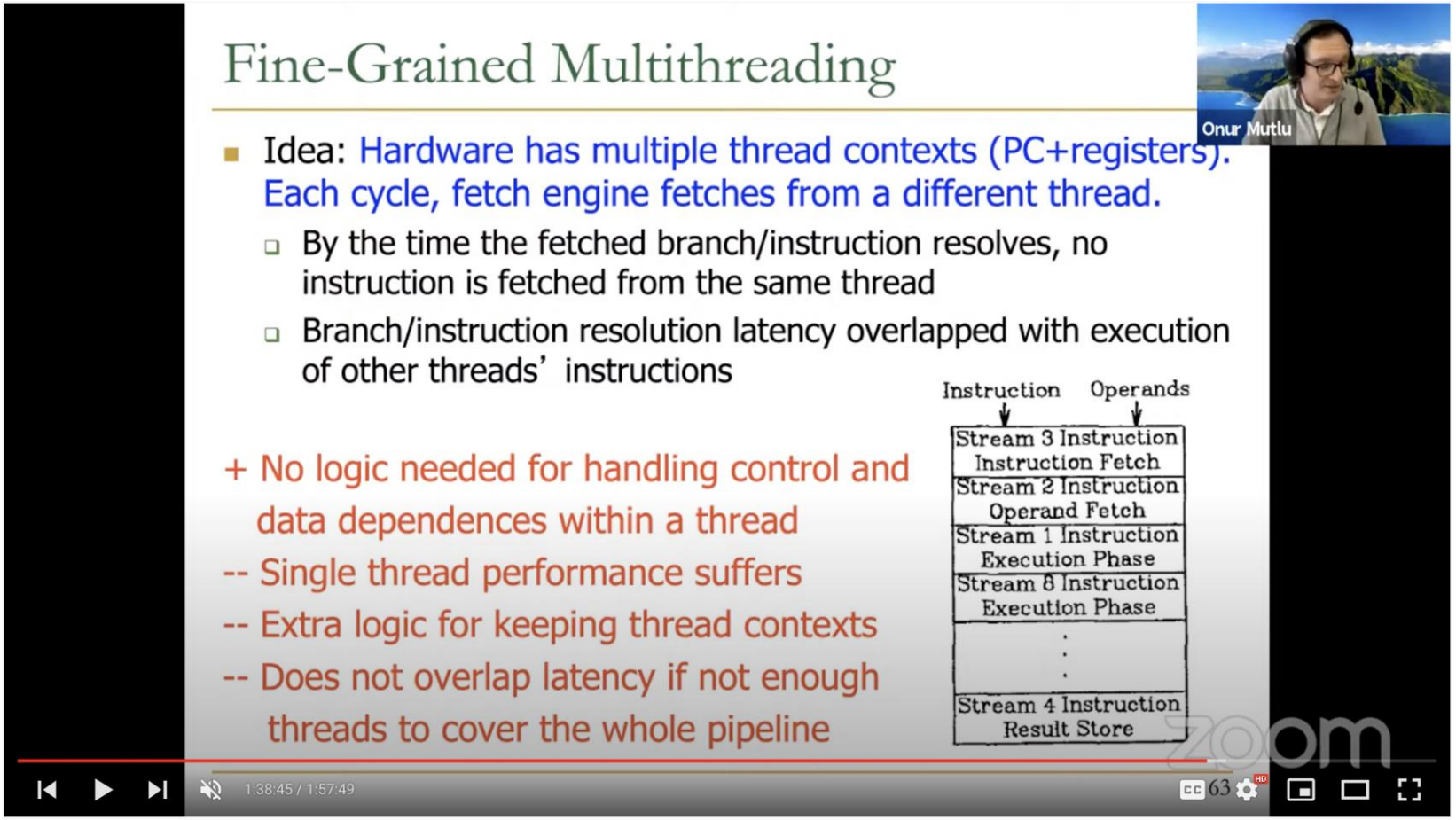
Main Programming Issues in Tightly-Coupled MP

- Load imbalance
  - How to partition a single task into multiple tasks
- Synchronization
  - How to synchronize (efficiently) between tasks
  - How to communicate between tasks
  - Locks, barriers, pipeline stages, condition variables, semaphores, atomic operations, …
- Contention (avoidance & management)
- Maximizing parallelism
- Ensuring correct operation while optimizing for performance

Aside: Hardware-based Multithreading

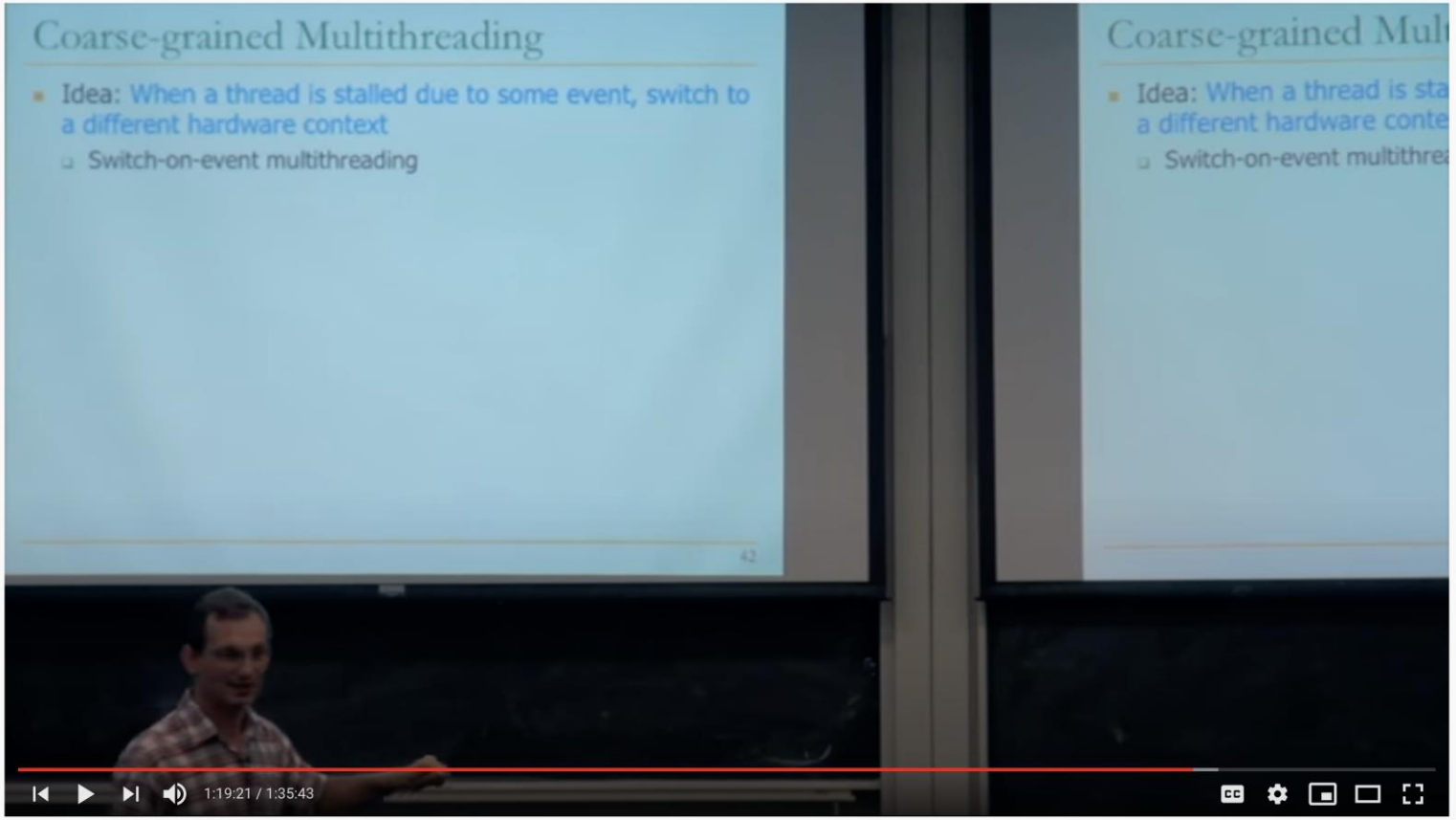
- Coarse grained
  - Quantum based
  - Event based (switch-on-event multithreading), e.g., switch on L3 miss
- Fine grained
  - Cycle by cycle
  - Thornton, "CDC 6600: Design of a Computer," 1970.
  - Burton Smith, "A pipelined, shared resource MIMD computer," ICPP 1978.
- Simultaneous
  - Can dispatch instructions from multiple threads at the same time
  - Good for improving execution unit utilization

Lecture on Fine-Grained Multithreading



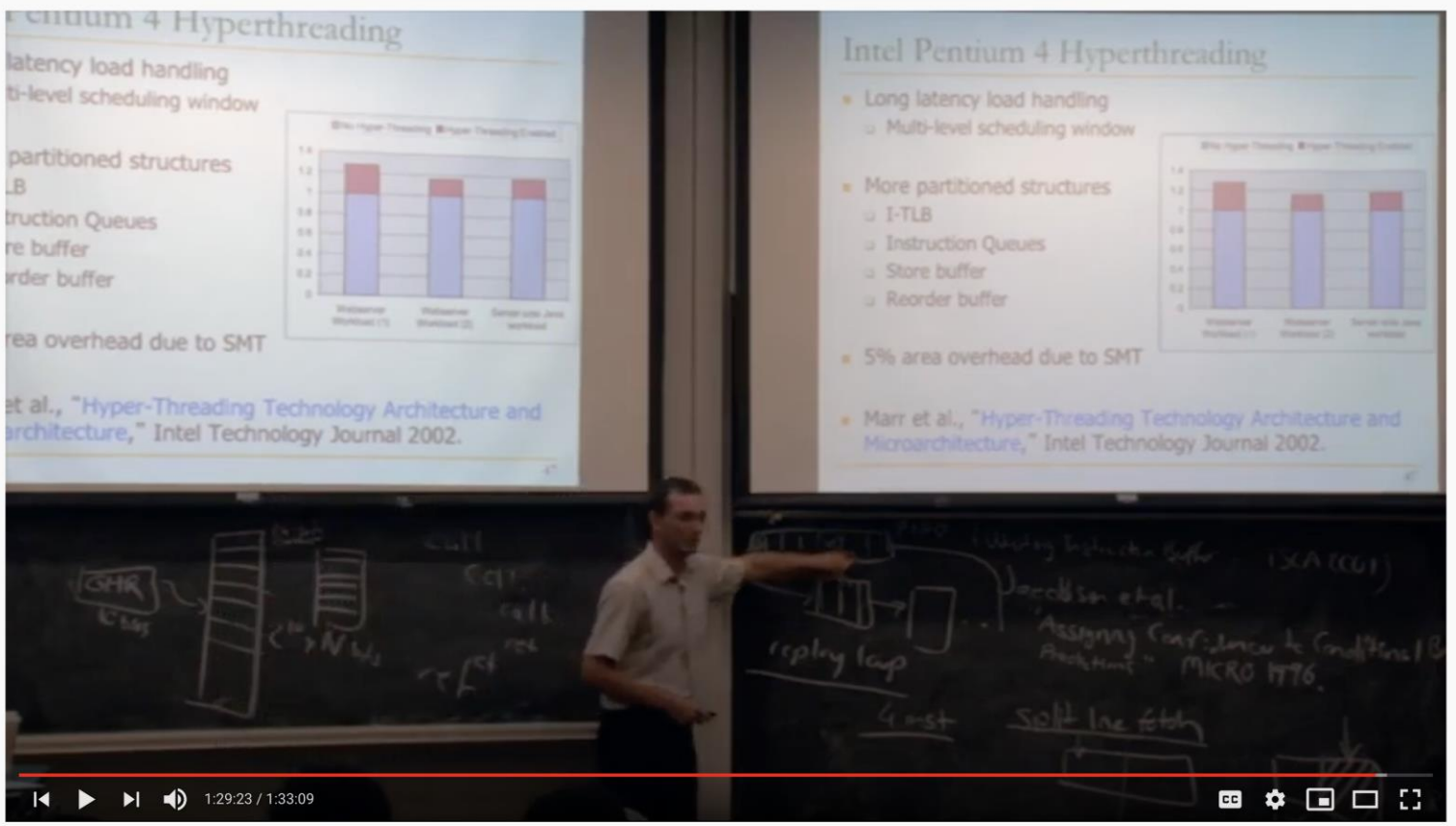
https://www.youtube.com/watch?v=6e5KZcCGBYw&list=PL5Q2soXY2Zi_uej3aY39YB5pfW4SJ7LlN&index=16

More on Multithreading (I)



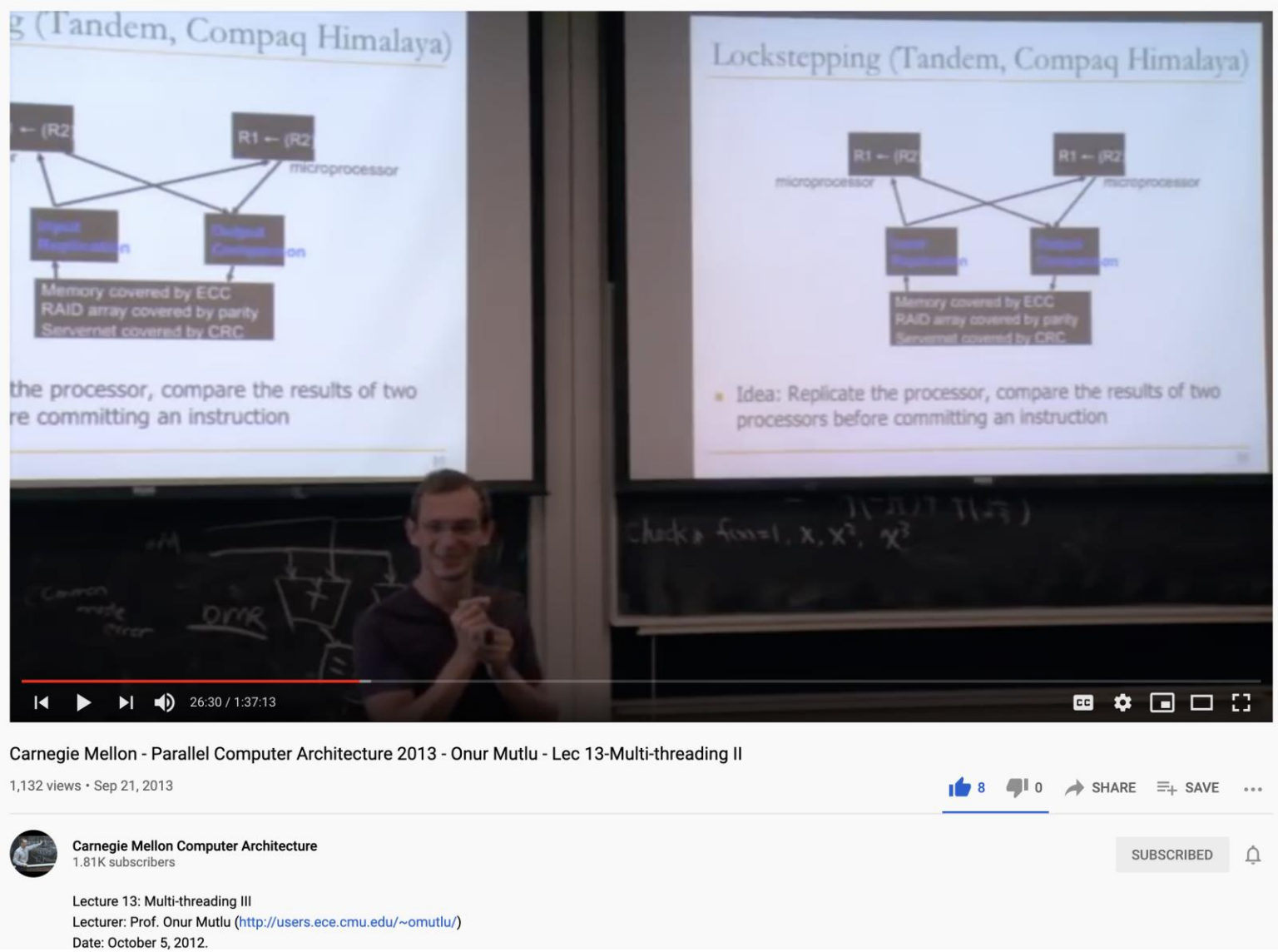
https://www.youtube.com/onurmutlulectures

More on Multithreading (II)



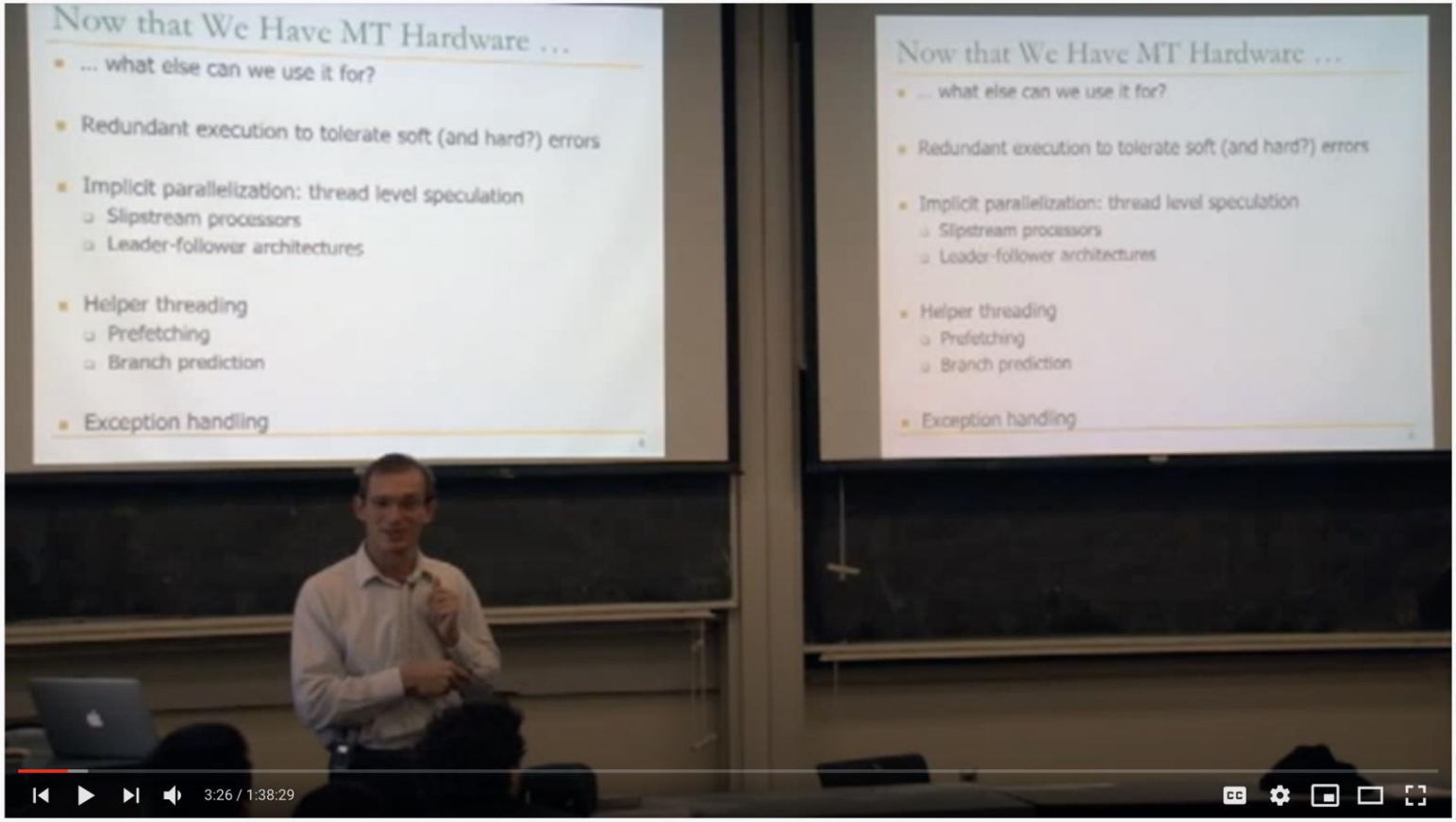
https://www.youtube.com/onurmutlulectures

More on Multithreading (III)



https://www.youtube.com/onurmutlulectures

More on Multithreading (IV)



https://www.youtube.com/onurmutlulectures

Lectures on Multithreading

- Parallel Computer Architecture, Fall 2012, Lecture 9
  - Multithreading I (CMU, Fall 2012)
  - https://www.youtube.com/watch?v=iqi9wFqFiNU&list=PL5PHm2jkkXmgDN1PLwOY_tGtUlynnyV6D&index=51
- Parallel Computer Architecture, Fall 2012, Lecture 10
  - Multithreading II (CMU, Fall 2012)
  - https://www.youtube.com/watch?v=e8lfl6MbILg&list=PL5PHm2jkkXmgDN1PLwOY_tGtUlynnyV6D&index=52
- Parallel Computer Architecture, Fall 2012, Lecture 13
  - Multithreading III (CMU, Fall 2012)
  - https://www.youtube.com/watch?v=7vkDpZ1-hHM&list=PL5PHm2jkkXmgDN1PLwOY_tGtUlynnyV6D&index=53
- Parallel Computer Architecture, Fall 2012, Lecture 15
  - Speculation I (CMU, Fall 2012)
  - https://www.youtube.com/watch?v=-hbmzIDe0sA&list=PL5PHm2jkkXmgDN1PLwOY_tGtUlynnyV6D&index=54

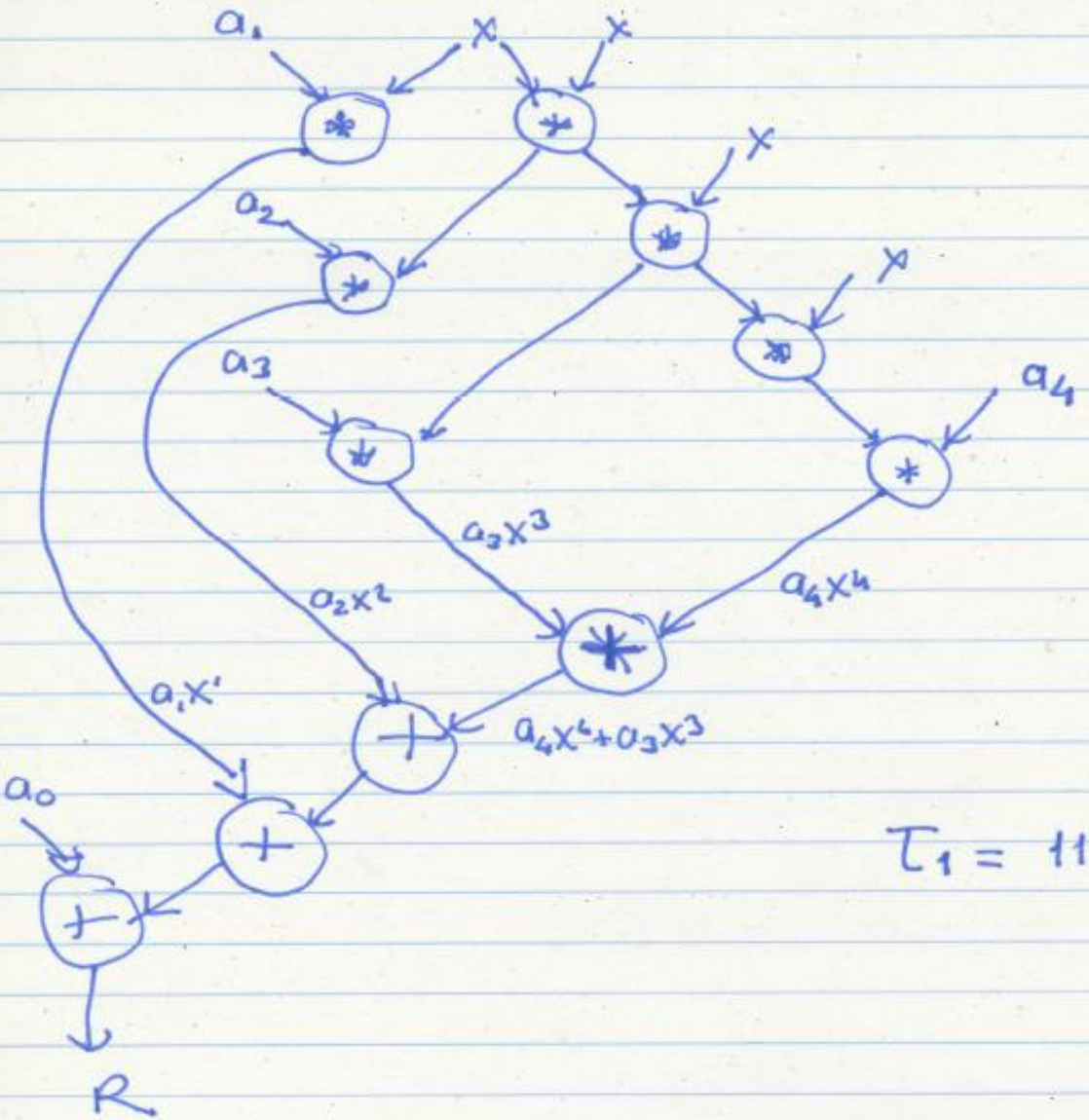
Limits of Parallel Speedup

Parallel Speedup Example

- $a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$
- Assume given inputs: x and each a_i
- Assume each operation 1 cycle, no communication cost, each op can be executed in a different processor
- How fast is this with a single processor?
  - Assume no pipelining or concurrent execution of instructions
- How fast is this with 3 processors?

$$R = a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$$

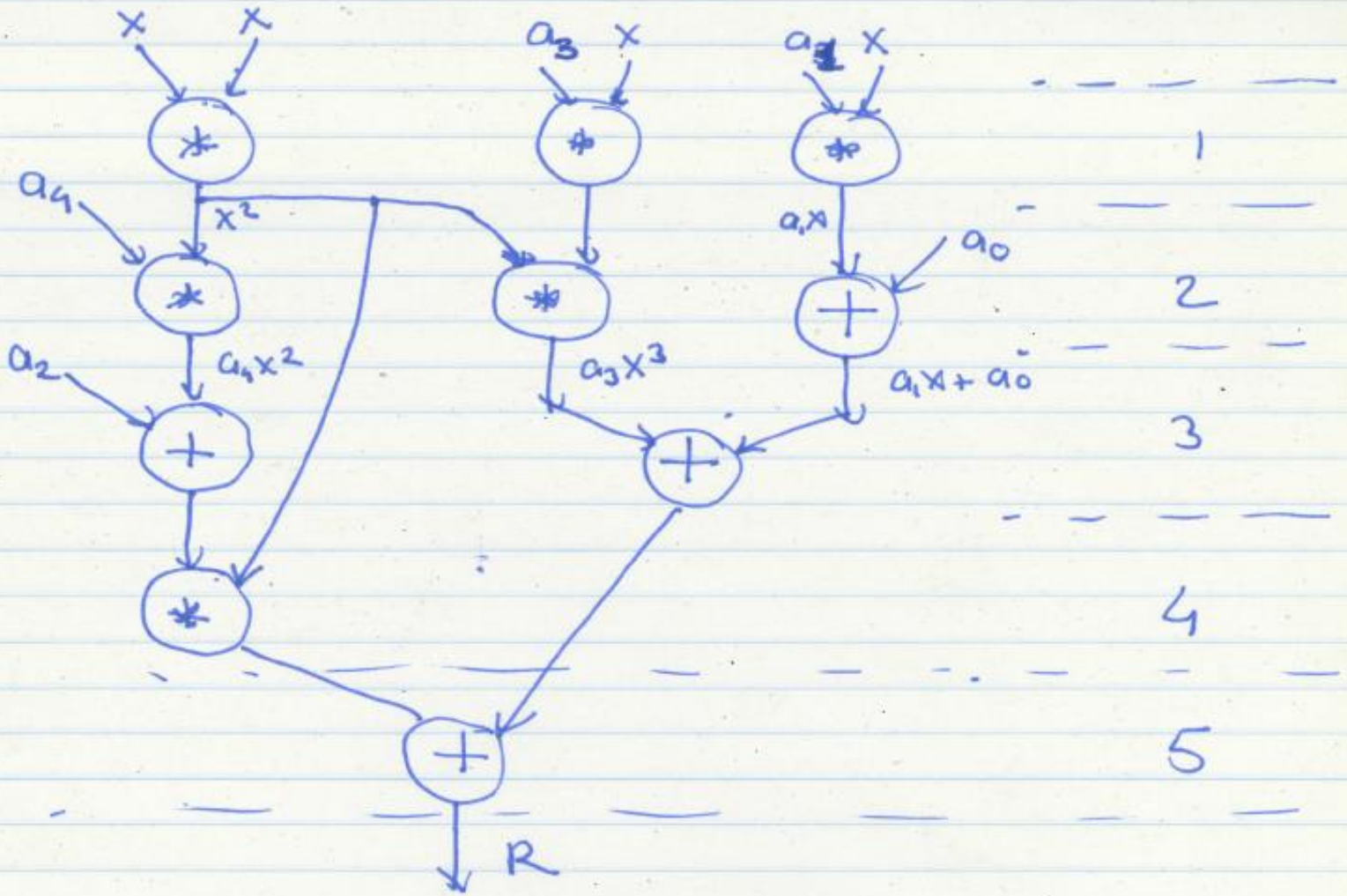
Single processor : 11 operations (draw the data flow graph)



$T_1 = 11$ cycles

$$R = a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$$

Three processors : T_3 (exec. time with 3 proc.)



$T_3 = 5$ cycles

Speedup with 3 Processors

$T_3 = 5$ cycles

Speedup with 3 processors $= \frac{11}{5} = 2.2$

$\left(\frac{T_1}{T_3}\right)$

Is this a fair comparison?

Revisiting the Single-Processor Algorithm

Revisit T_1

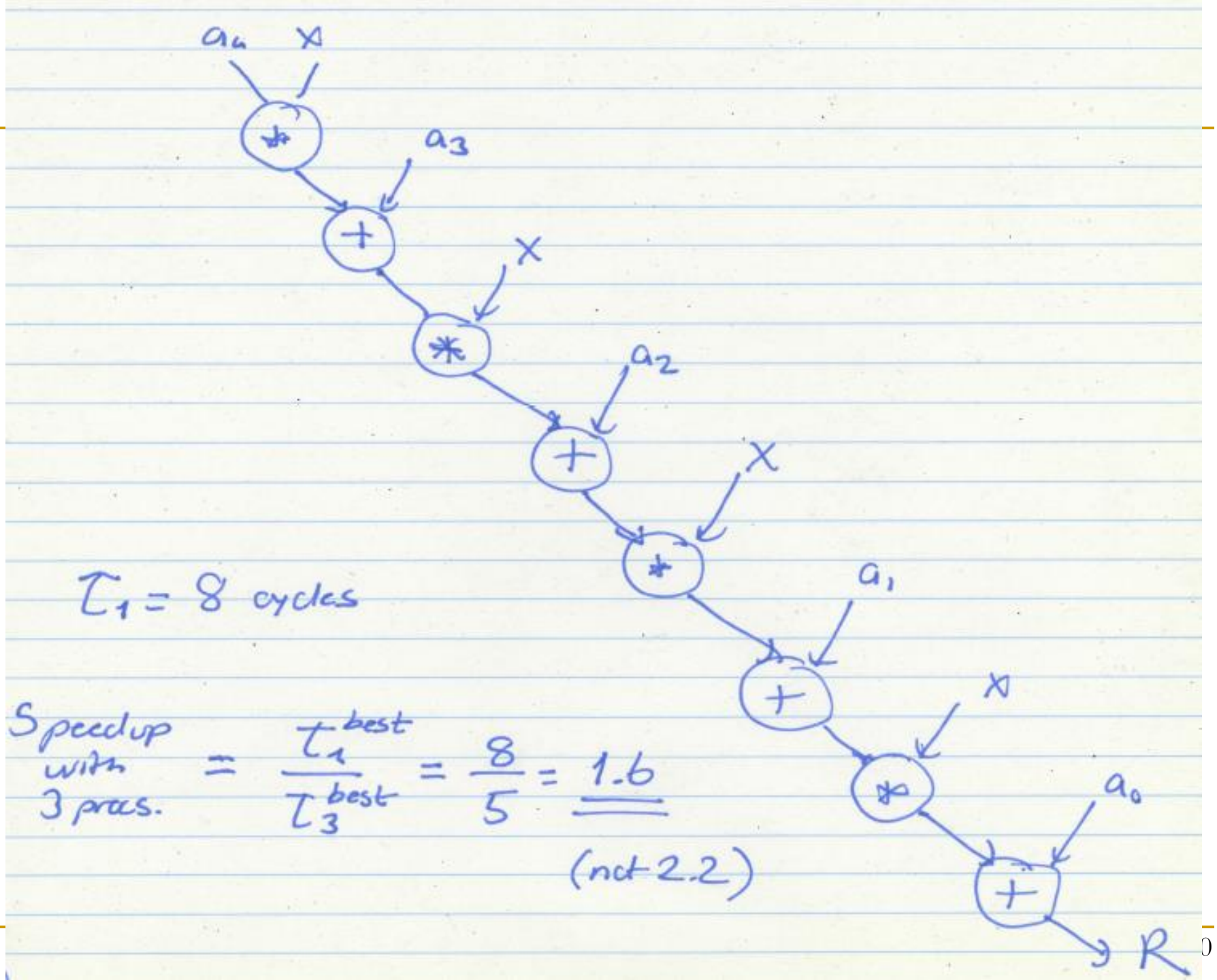
Better single-processor algorithm:

$$R = a_4 x^4 + a_3 x^3 + a_2 x^2 + a_1 x + a_0$$

$$R = (((a_4 x + a_3) x + a_2) x + a_1) x + a_0$$

(Horner's method)

Horner, "A new method of solving numerical equations of all orders, by continuous approximation," Philosophical Transactions of the Royal Society, 1819.



$T_1 = 8$ cycles

Speedup with 3 procs. $= \frac{T_1^{best}}{T_3^{best}} = \frac{8}{5} = \underline{\underline{1.6}}$

(not 2.2)

Superlinear Speedup

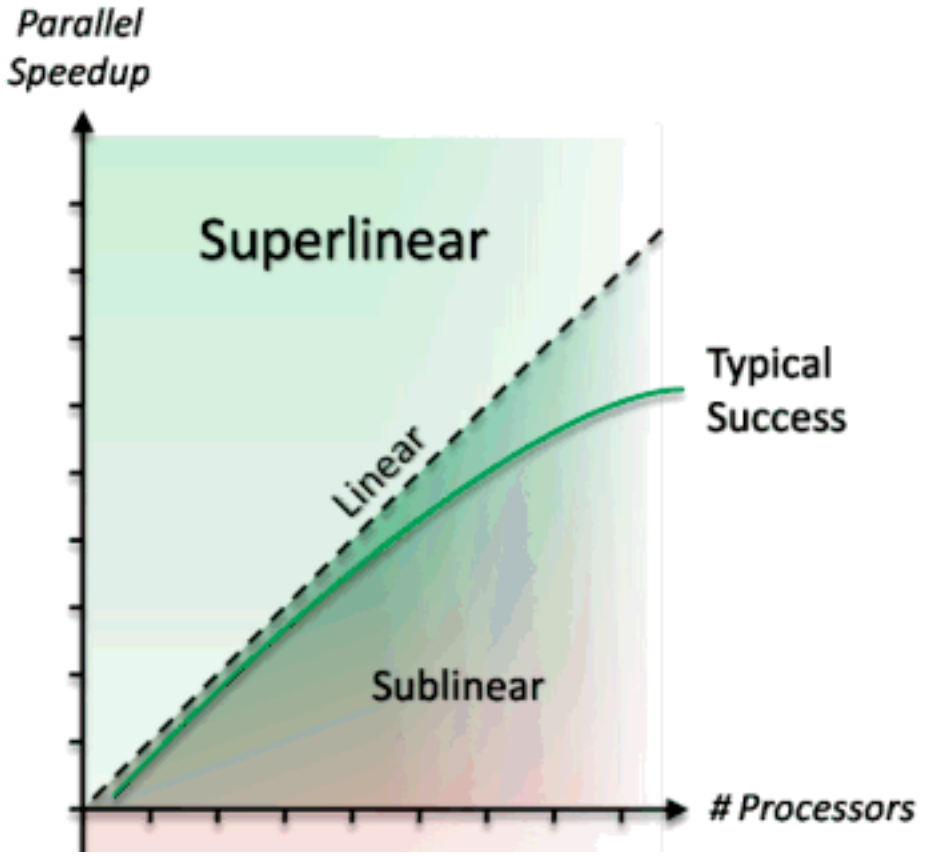
- Can speedup be greater than P with P processing elements?

- Unfair comparisons

  Compare best parallel algorithm to wimpy serial algorithm → unfair

- Cache/memory effects

  More processors → more cache or memory → fewer misses in cache/mem



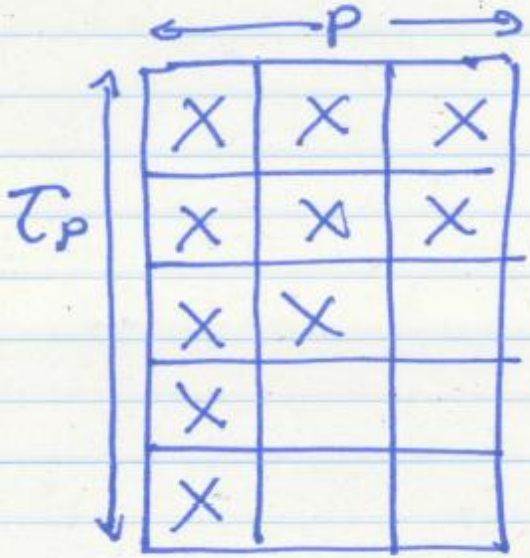
Utilization, Redundancy, Efficiency

- Traditional metrics
  - Assume all P processors are tied up for parallel computation
- Utilization: How much processing capability is used
  - U = (# Operations in parallel version) / (processors x Time)
- Redundancy: how much extra work is done with parallel processing
  - R = (# of operations in parallel version) / (# operations in best single processor algorithm version)
- Efficiency
  - E = (Time with 1 processor) / (processors x Time with P processors)
  - E = U/R

Utilization of a Multiprocessor

Multiprocessor metrics

Utilization : How much processing capability we use



$$U = \frac{\text{Ops with p proc.}}{p \times T_p}$$

$$U = \frac{10 \text{ operations (in parallel version)}}{3 \text{ processors} \times 5 \text{ time units}}$$

$$= \frac{10}{15}$$

Redundancy: How much extra work due to multiprocessing

$$R = \frac{\text{Ops with p proc.}^{best}}{\text{Ops with 1 proc.}^{best}} = \frac{10}{8}$$

R is always ≥ 1

Efficiency: How much resource we use compared to how much resource we can get away with

$$E = \frac{1 \cdot T_1^{best}}{p \cdot T_p^{best}}$$

(tying up 1 proc for T_p time units)

(tying up p proc for T_p time units)

$$= \frac{8}{15} \qquad \left(E = \frac{U}{R} \right)$$

Amdahl's Law and Caveats of Parallelism

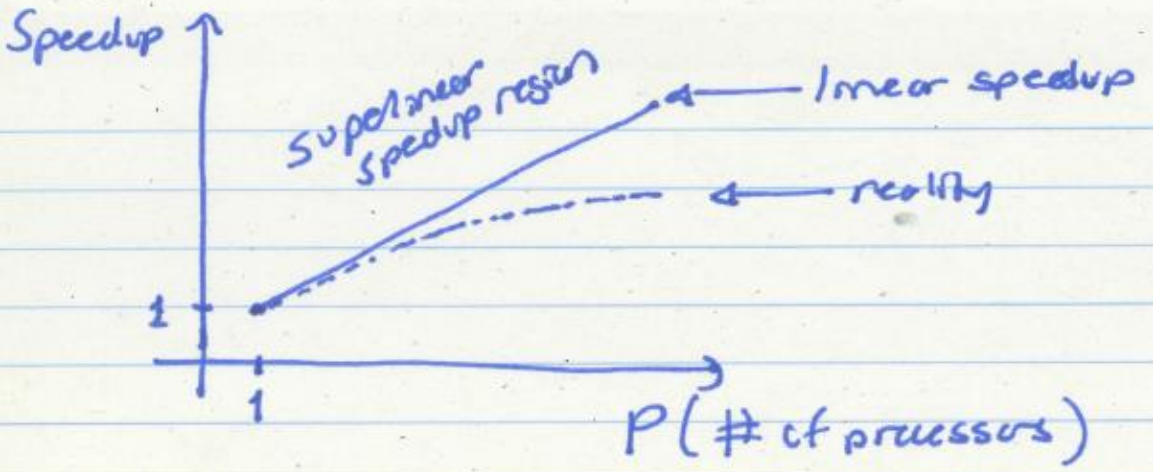
Amdahl's Law

- Amdahl's Law
  - f: Parallelizable fraction of a program
  - N: Number of processors

$$\text{Speedup} = \frac{1}{1 - f + \frac{f}{N}}$$

  - Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS 1967.
- Maximum speedup limited by serial portion: Serial bottleneck

Caveats of Parallelism (I)



Why the reality? (diminishing returns)

$$T_p = \alpha \cdot \frac{T_1}{p} + (1-\alpha) \cdot T_1$$

α: parallelizable part/fraction of the single-processor program

$(1-\alpha)$: non-parallelizable part

Amdahl's Law

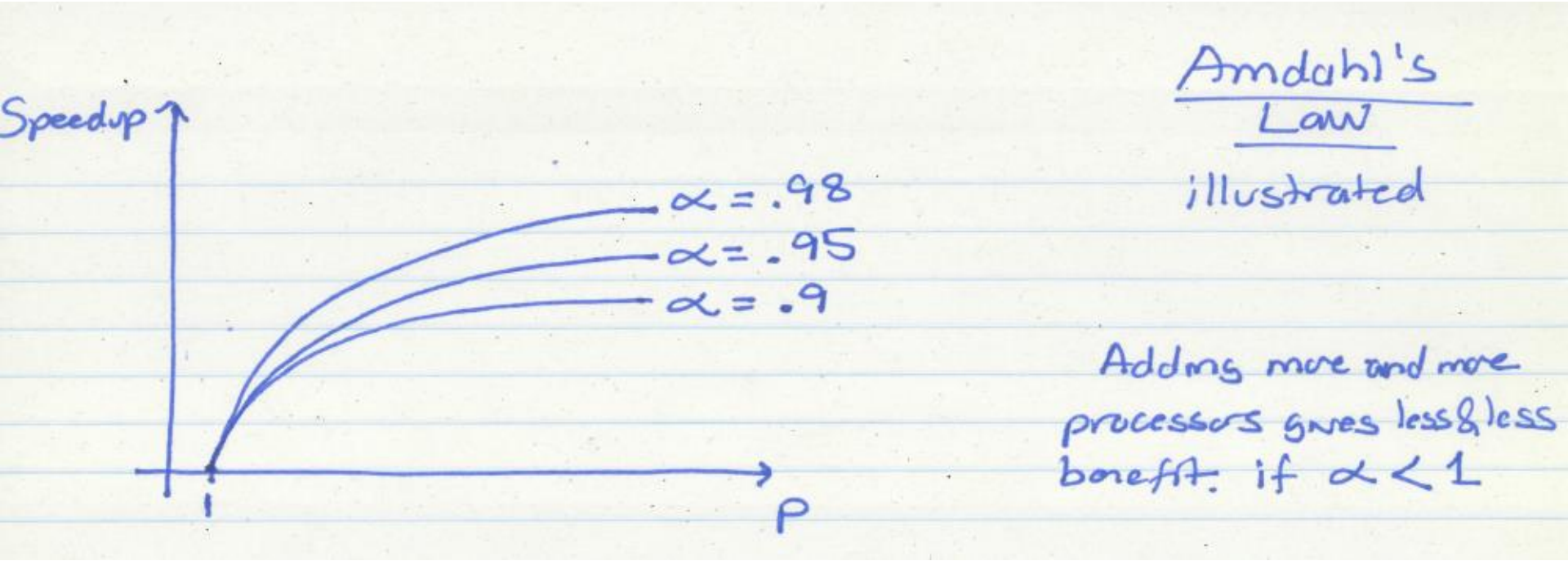
$$\text{Speedup with } p \text{ proc.} = \frac{T_1}{T_p} = \frac{1}{\frac{\alpha}{p} + (1-\alpha)}$$

$$\text{Speedup as } p \to \infty = \frac{1}{1-\alpha}$$

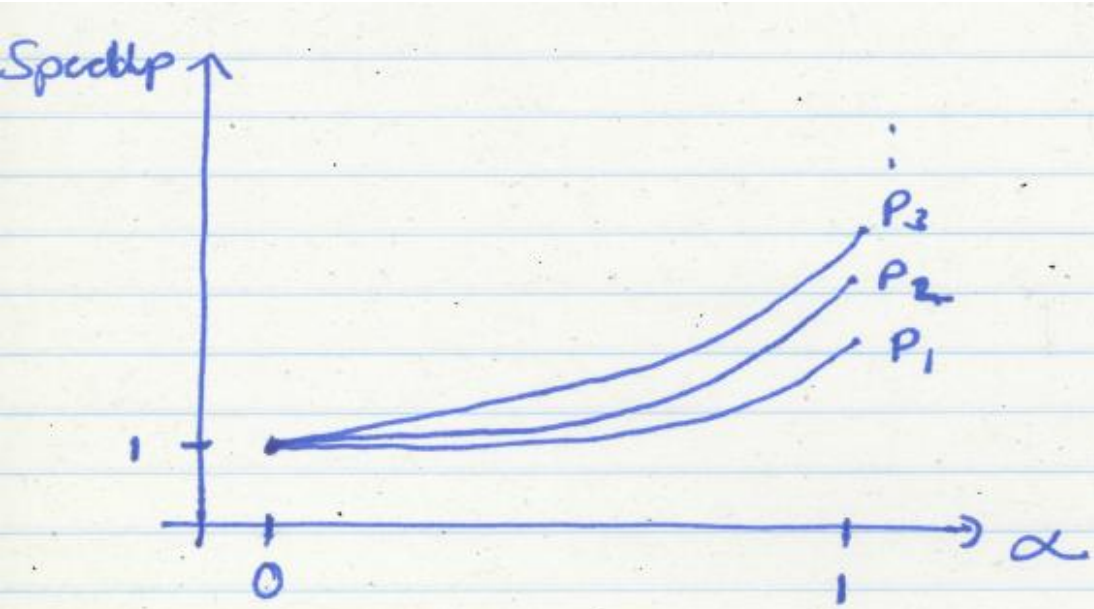
$1-\alpha$ → bottleneck for parallel Speedup

Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS 1967.

Amdahl's Law Implication 1



Amdahl's Law Implication 2



The benefit (speedup) is small until $\alpha \approx 1$

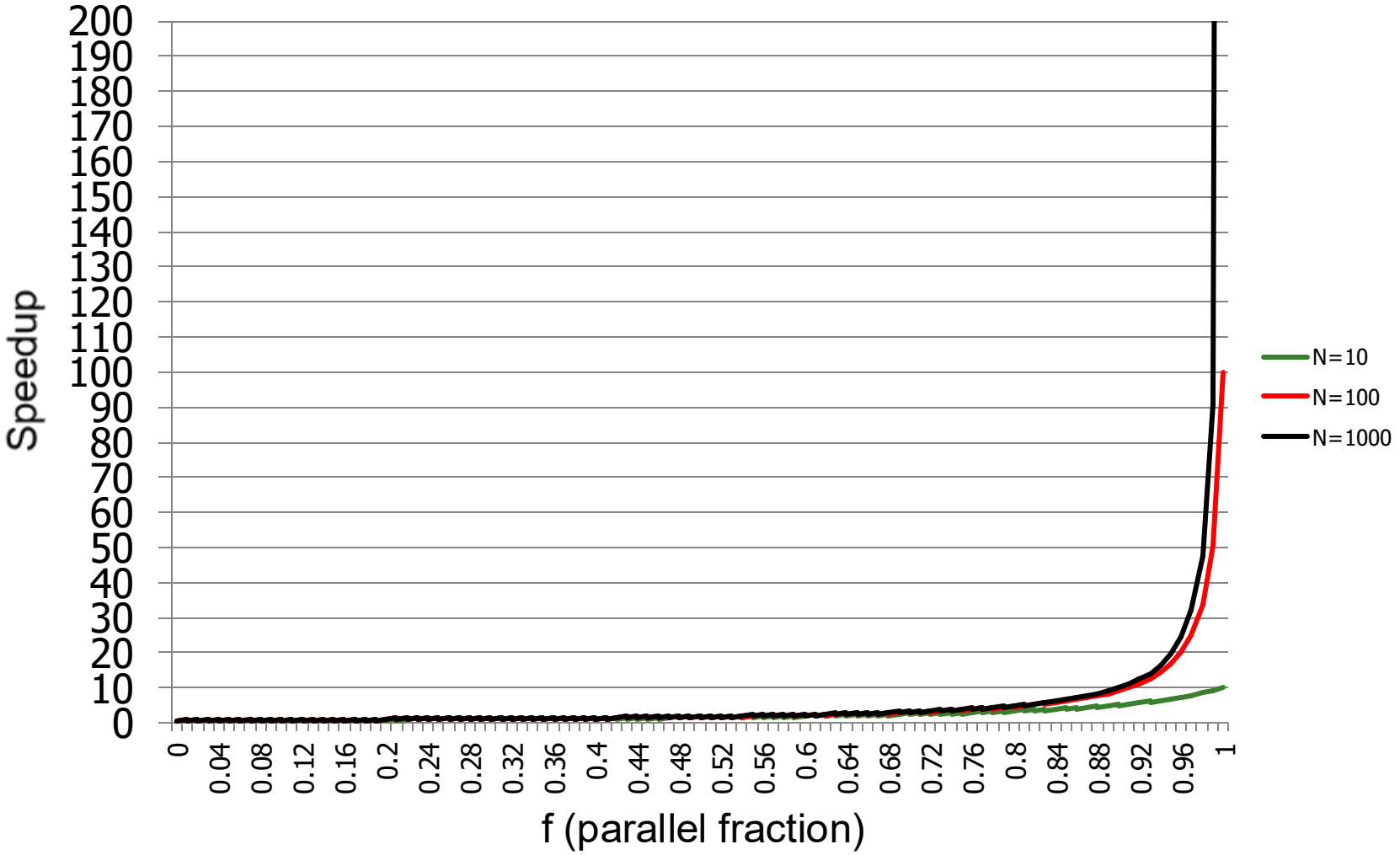
Caveats of Parallelism (II)

- Amdahl's Law
  - f: Parallelizable fraction of a program
  - N: Number of processors

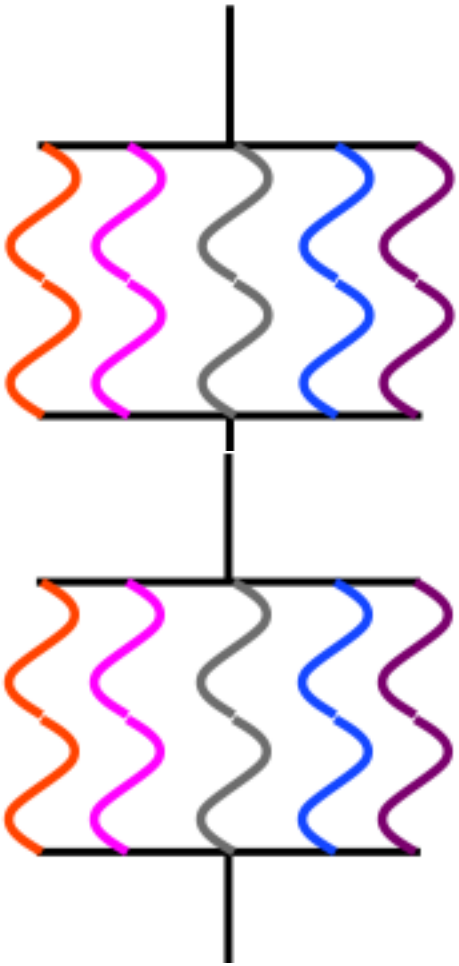
$$\text{Speedup} = \frac{1}{(1 - f) + \frac{f}{N}}$$

  - Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS 1967.
- Maximum speedup limited by serial portion: Serial bottleneck
- Parallel portion is usually not perfectly parallel
  - Synchronization overhead (e.g., updates to shared data)
  - Load imbalance overhead (imperfect parallelization)
  - Resource sharing overhead (contention among N processors)

Sequential Bottleneck



Why the Sequential Bottleneck?

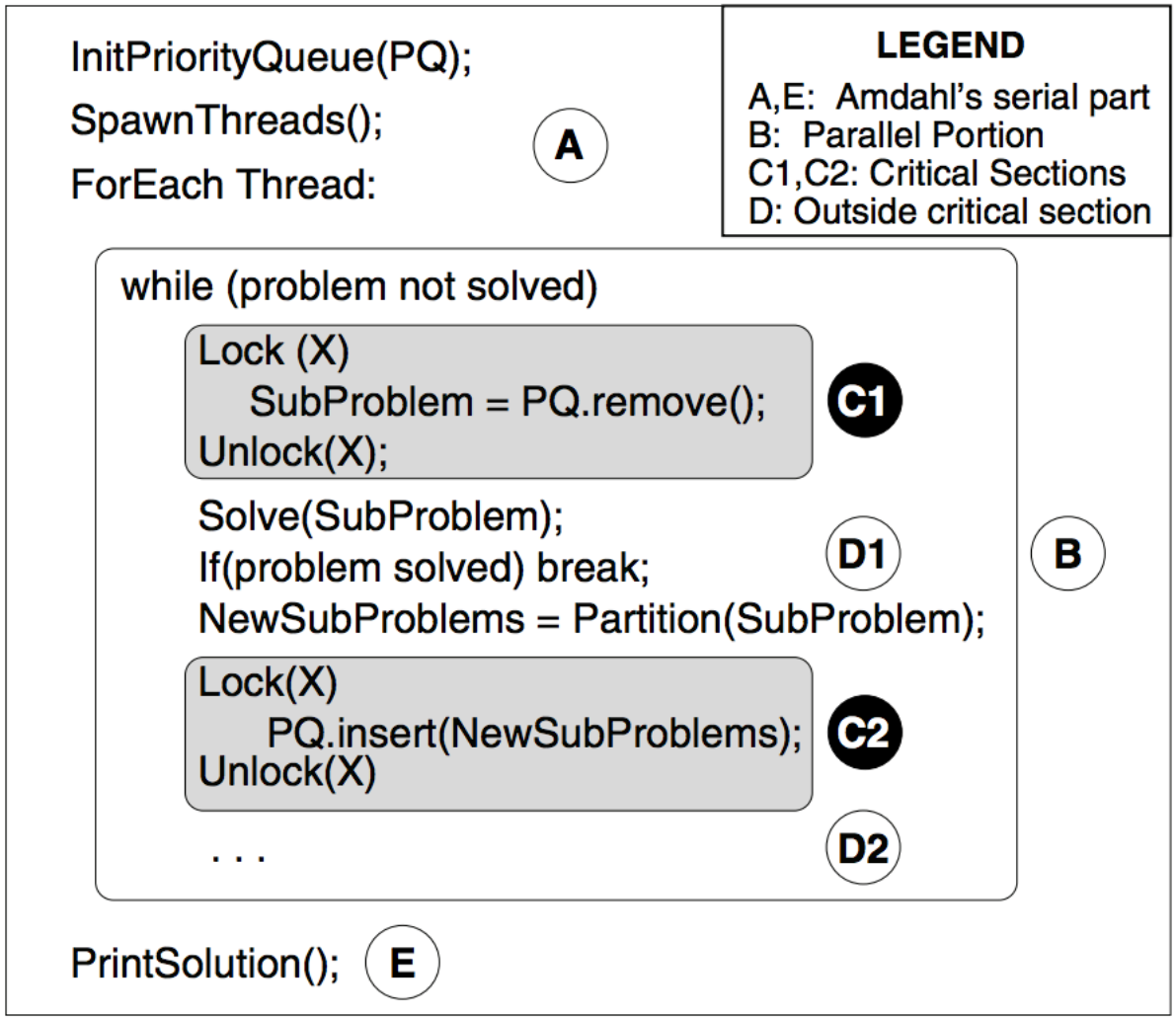


- Parallel machines have the sequential bottleneck
- Main cause: Non-parallelizable operations on data (e.g. non-parallelizable loops)

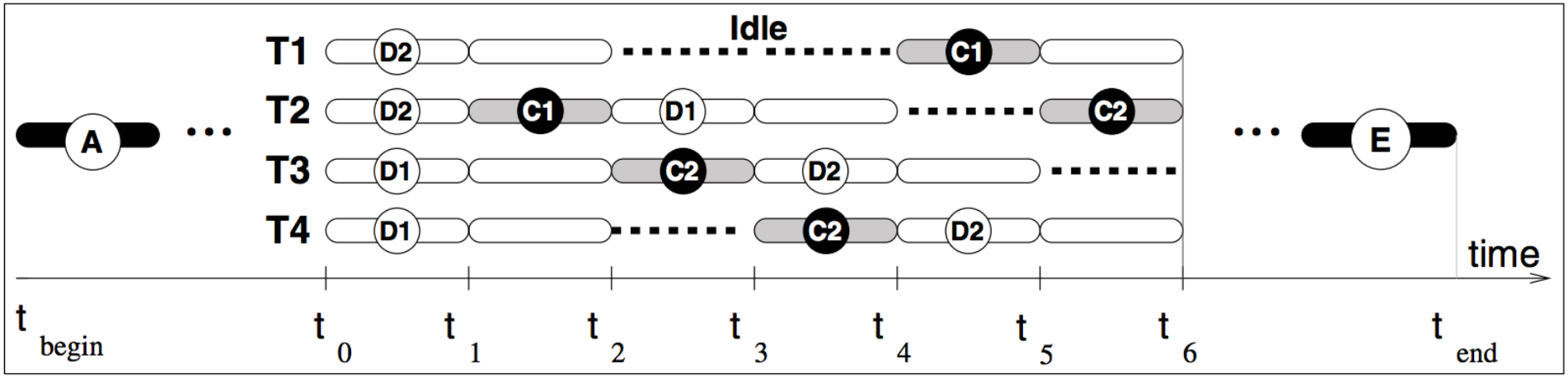
```
for ( i = 0 ; i < N; i++)
    A[i] = (A[i] + A[i-1]) / 2
```

- There are other causes as well:
  - Single thread prepares data and spawns parallel tasks (usually sequential)

Another Example of Sequential Bottleneck (I)



Another Example of Sequential Bottleneck (II)



Suleman+, "Accelerating Critical Section Execution with Asymmetric Multi-Core Architectures," ASPLOS 2009.

Bottlenecks in Parallel Portion

- Synchronization: Operations manipulating shared data cannot be parallelized
  - Locks, mutual exclusion, barrier synchronization
  - Communication: Tasks may need values from each other

  - Causes thread serialization when shared data is contended

- Load Imbalance: Parallel tasks may have different lengths
  - Due to imperfect parallelization or microarchitectural effects

  - Reduces speedup in parallel portion

- Resource Contention: Parallel tasks can share hardware resources, delaying each other
  - Replicating all resources (e.g., memory) expensive

  - Additional latency not present when each task runs alone

Bottlenecks in Parallel Portion: Another View

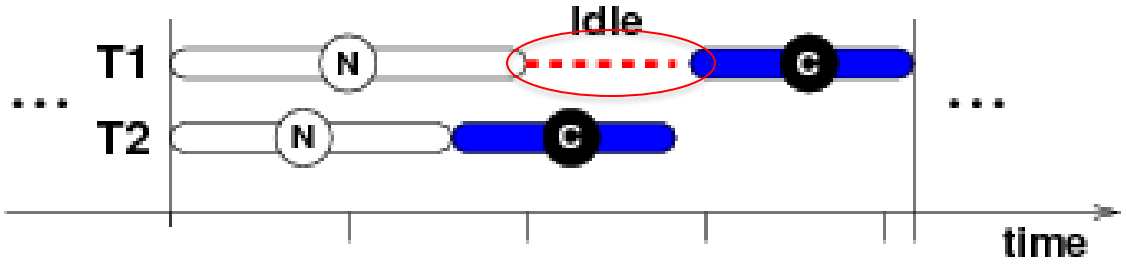
- Threads in a multi-threaded application can be inter-dependent
  - As opposed to threads from different applications
- Such threads can synchronize with each other
  - Locks, barriers, pipeline stages, condition variables, semaphores, …
- Some threads can be on the critical path of execution due to synchronization; some threads are not
- Within a thread, some "code segments" may be on the critical path of execution; some are not

Critical Sections

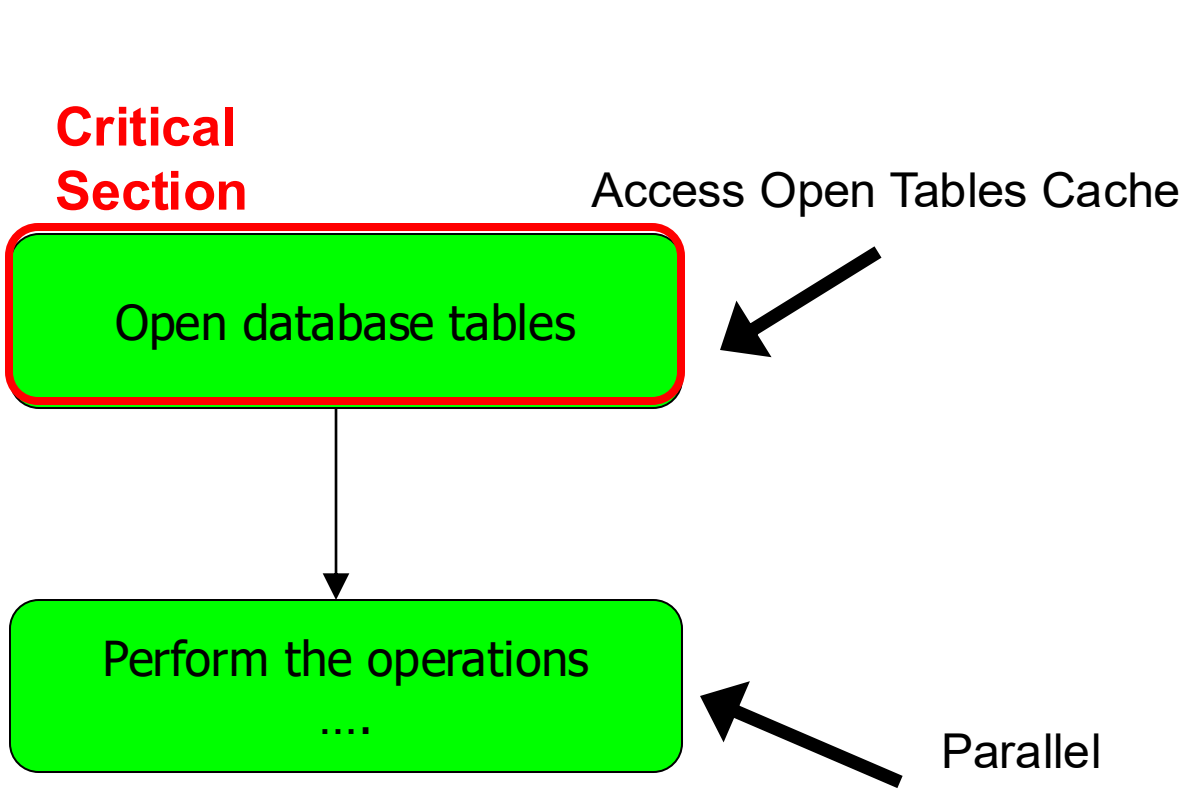
- Enforce mutually exclusive access to shared data
- Only one thread can be executing it at a time
- Contended critical sections make threads wait → threads causing serialization can be on the critical path

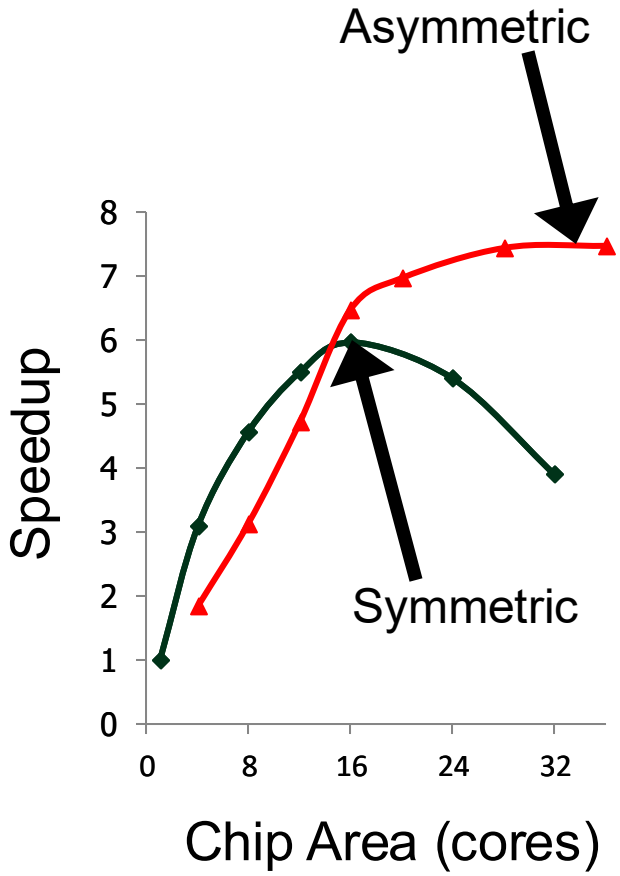
Each thread:

```
loop {
    Compute                  N
    lock(A)
        Update shared data
    unlock(A)                C
}
```



Critical Section Example from MySQL

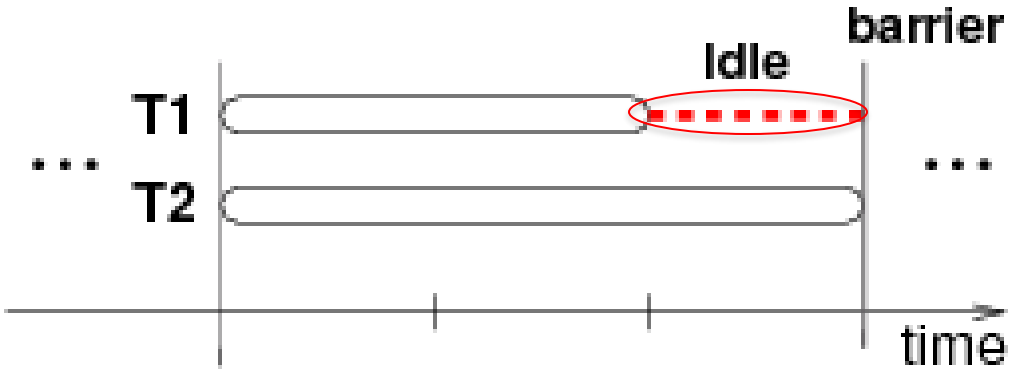




Barriers

- Synchronization point
- Threads have to wait until all threads reach the barrier
- Last thread arriving to the barrier is on the critical path

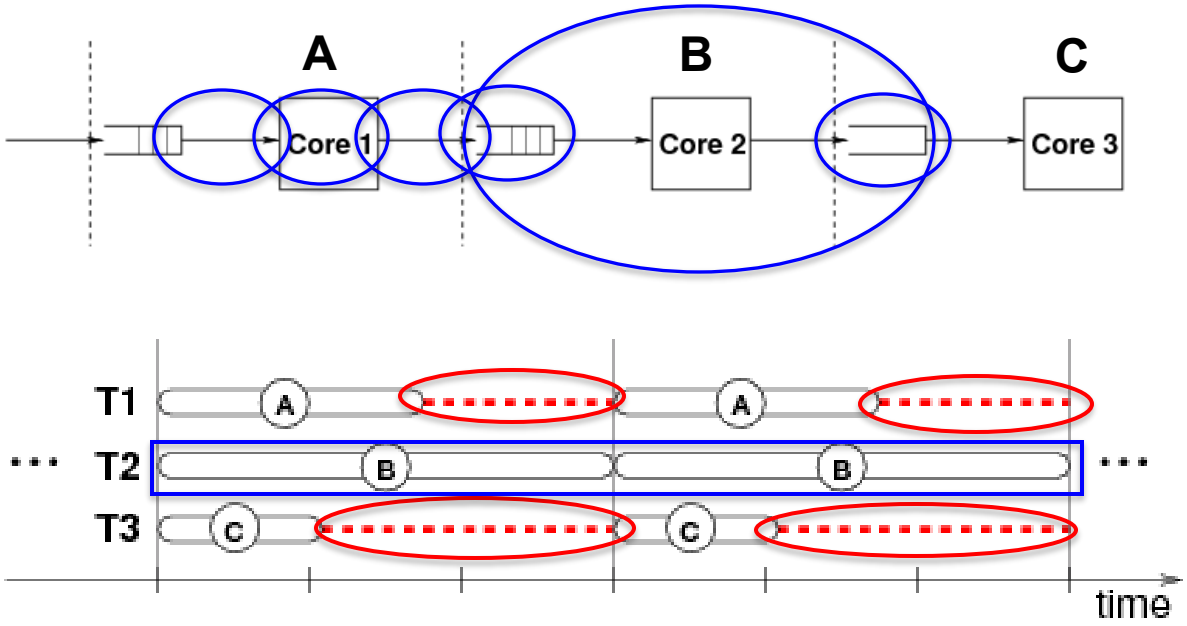
```
Each thread:
    loop1 {
        Compute
    }
    barrier
    loop2 {
        Compute
    }
```



Stages of Pipelined Programs

- Loop iterations are statically divided into code segments called *stages*
- Threads execute stages on different cores
- Thread executing the slowest stage is on the critical path

```
loop {
  Compute1   A
  Compute2   B
  Compute3   C
}
```



Difficulty in Parallel Programming

- Little difficulty if parallelism is natural
  - "Embarrassingly parallel" applications
  - Multimedia, physical simulation, graphics
  - Large web servers, databases?

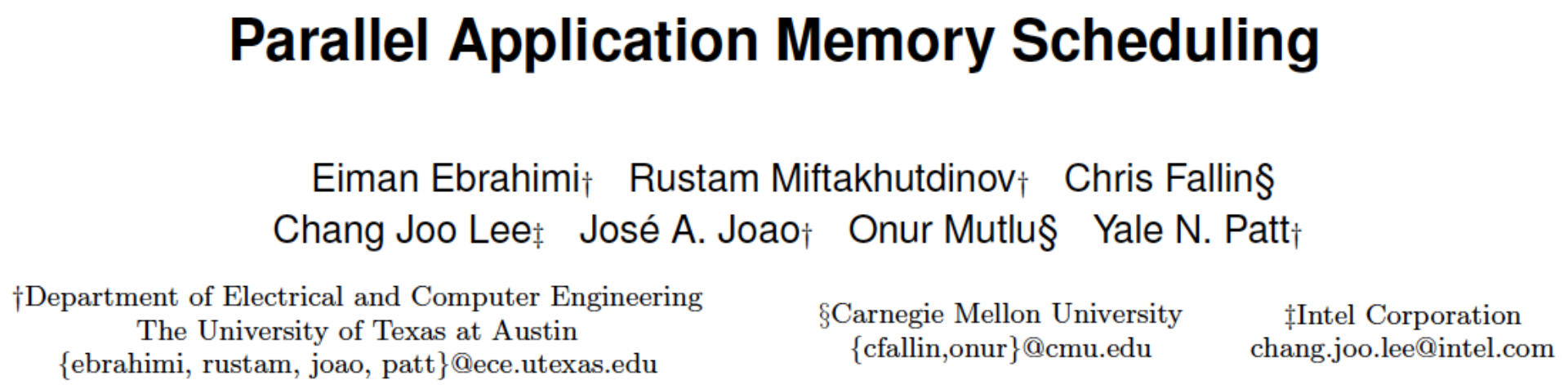
- Difficulty is in
  - Getting parallel programs to work correctly
  - Optimizing performance in the presence of bottlenecks

- Much of **parallel computer architecture** is about
  - Designing machines that overcome the sequential and parallel bottlenecks to achieve higher performance and efficiency
  - Making programmer's job easier in writing correct and high-performance parallel programs

Some Readings on Bottlenecks & Bottleneck Acceleration

Parallel Application Memory Scheduling

- Eiman Ebrahimi, Rustam Miftakhutdinov, Chris Fallin, Chang Joo Lee, Onur Mutlu, and Yale N. Patt,
  "Parallel Application Memory Scheduling"
  *Proceedings of the 44th International Symposium on Microarchitecture (**MICRO**)*, Porto Alegre, Brazil, December 2011. Slides (pptx)



Parallel Application Memory Scheduling

Eiman Ebrahimi† Rustam Miftakhutdinov† Chris Fallin§
Chang Joo Lee‡ José A. Joao† Onur Mutlu§ Yale N. Patt†

†Department of Electrical and Computer Engineering
The University of Texas at Austin
{ebrahimi, rustam, joao, patt}@ece.utexas.edu

§Carnegie Mellon University
{cfallin,onur}@cmu.edu

‡Intel Corporation
chang.joo.lee@intel.com

Accelerated Critical Sections

- M. Aater Suleman, Onur Mutlu, Moinuddin K. Qureshi, and Yale N. Patt, **"Accelerating Critical Section Execution with Asymmetric Multi-Core Architectures"**
  *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (**ASPLOS**)*, pages 253-264, Washington, DC, March 2009. Slides (ppt)
  One of the 13 computer architecture papers of 2009 selected as Top Picks by IEEE Micro.

Accelerating Critical Section Execution with Asymmetric Multi-Core Architectures

M. Aater Suleman
University of Texas at Austin
suleman@hps.utexas.edu

Onur Mutlu
Carnegie Mellon University
onur@cmu.edu

Moinuddin K. Qureshi
IBM Research
mkquresh@us.ibm.com

Yale N. Patt
University of Texas at Austin
patt@ece.utexas.edu

Bottleneck Identification & Scheduling

- Jose A. Joao, M. Aater Suleman, Onur Mutlu, and Yale N. Patt, **"Bottleneck Identification and Scheduling in Multithreaded Applications"**
  *Proceedings of the 17th International Conference on Architectural Support for Programming Languages and Operating Systems (**ASPLOS**)*, London, UK, March 2012. Slides (ppt) (pdf)

Bottleneck Identification and Scheduling in Multithreaded Applications

José A. Joao
ECE Department
The University of Texas at Austin
joao@ece.utexas.edu

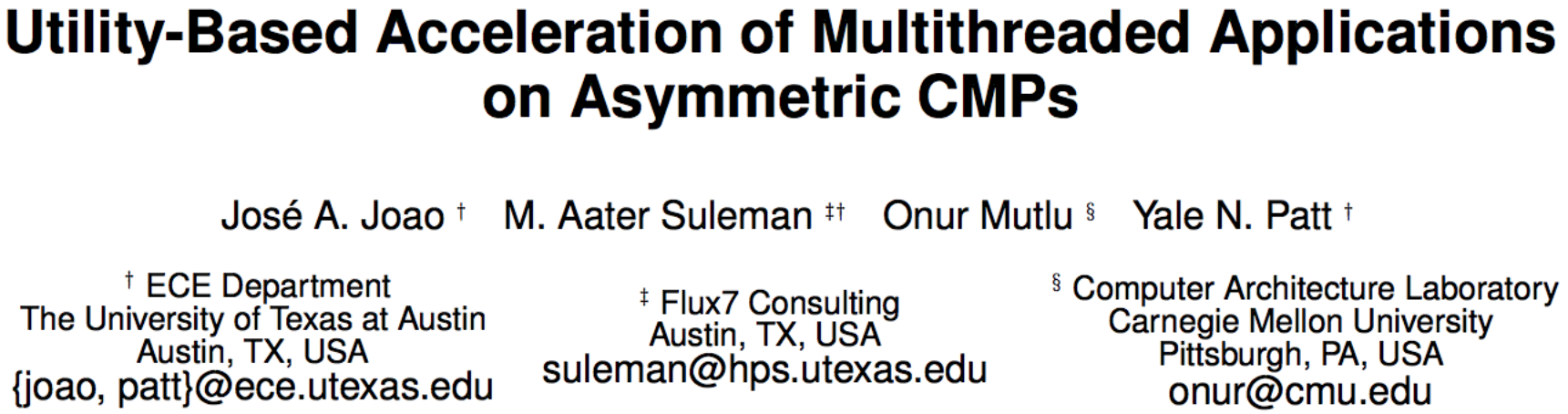
M. Aater Suleman
Calxeda Inc.
aater.suleman@calxeda.com

Onur Mutlu
Computer Architecture Lab.
Carnegie Mellon University
onur@cmu.edu

Yale N. Patt
ECE Department
The University of Texas at Austin
patt@ece.utexas.edu

Utility-Based Acceleration

- Jose A. Joao, M. Aater Suleman, Onur Mutlu, and Yale N. Patt, **"Utility-Based Acceleration of Multithreaded Applications on Asymmetric CMPs"**
  *Proceedings of the 40th International Symposium on Computer Architecture (**ISCA**)*, Tel-Aviv, Israel, June 2013. Slides (ppt) Slides (pdf)



Utility-Based Acceleration of Multithreaded Applications on Asymmetric CMPs

José A. Joao † M. Aater Suleman ‡† Onur Mutlu § Yale N. Patt †

† ECE Department
The University of Texas at Austin
Austin, TX, USA
{joao, patt}@ece.utexas.edu

‡ Flux7 Consulting
Austin, TX, USA
suleman@hps.utexas.edu

§ Computer Architecture Laboratory
Carnegie Mellon University
Pittsburgh, PA, USA
onur@cmu.edu

Data Marshaling

- M. Aater Suleman, Onur Mutlu, Jose A. Joao, Khubaib, and Yale N. Patt, **"Data Marshaling for Multi-core Architectures"** *Proceedings of the 37th International Symposium on Computer Architecture (**ISCA**)*, pages 441-450, Saint-Malo, France, June 2010. Slides (ppt)
  One of the 11 computer architecture papers of 2010 selected as Top Picks by IEEE Micro.

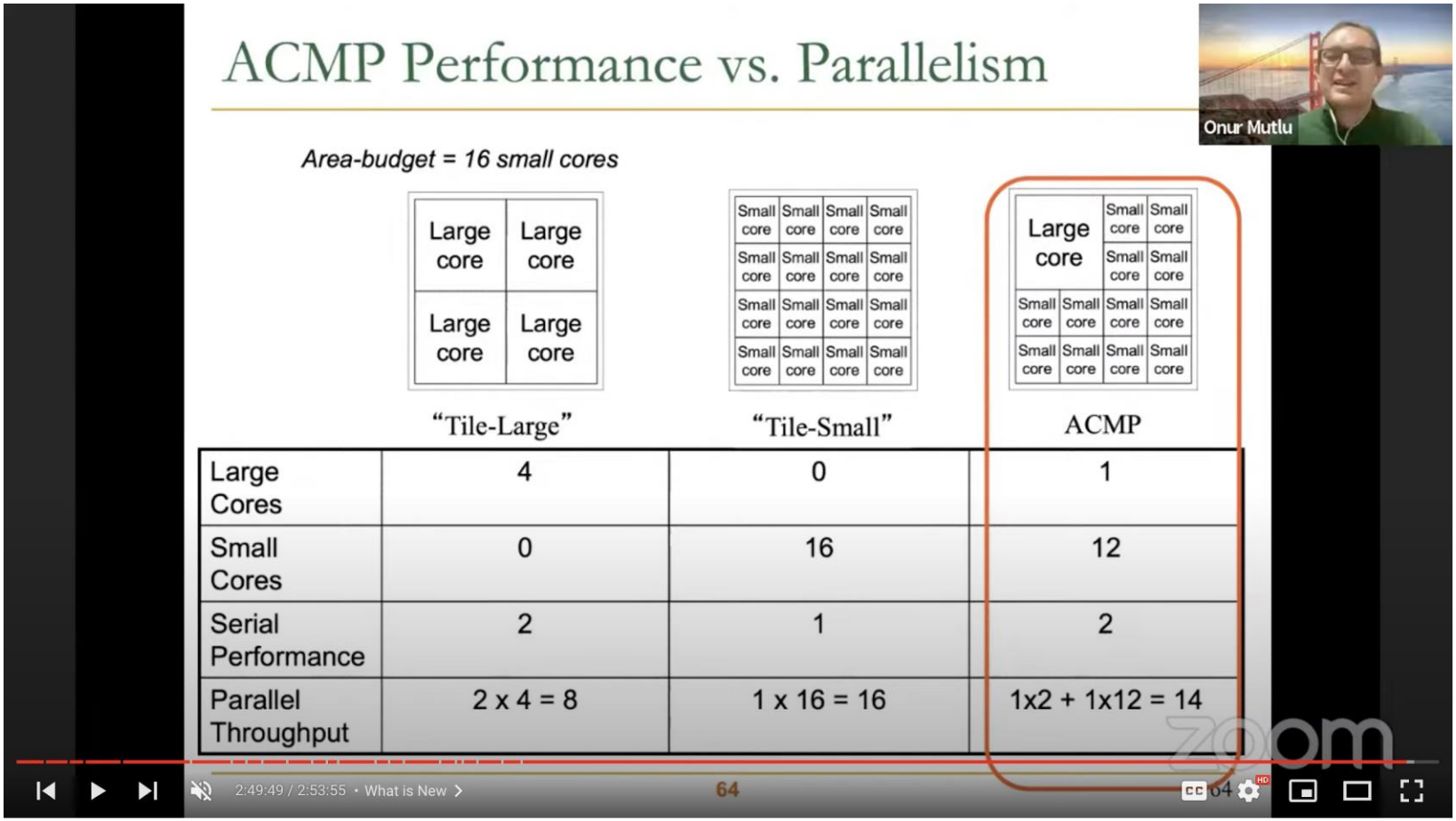
Data Marshaling for Multi-core Architectures

M. Aater Suleman† Onur Mutlu§ José A. Joao† Khubaib† Yale N. Patt†

†The University of Texas at Austin
{suleman, joao, khubaib, patt}@hps.utexas.edu

§Carnegie Mellon University
onur@cmu.edu

Lecture on Parallelism & Heterogeneity



Livestream - Computer Architecture - ETH Zürich (Fall 2021)

Computer Architecture - Lecture 17: Parallelism & Heterogeneity (Fall 2021)

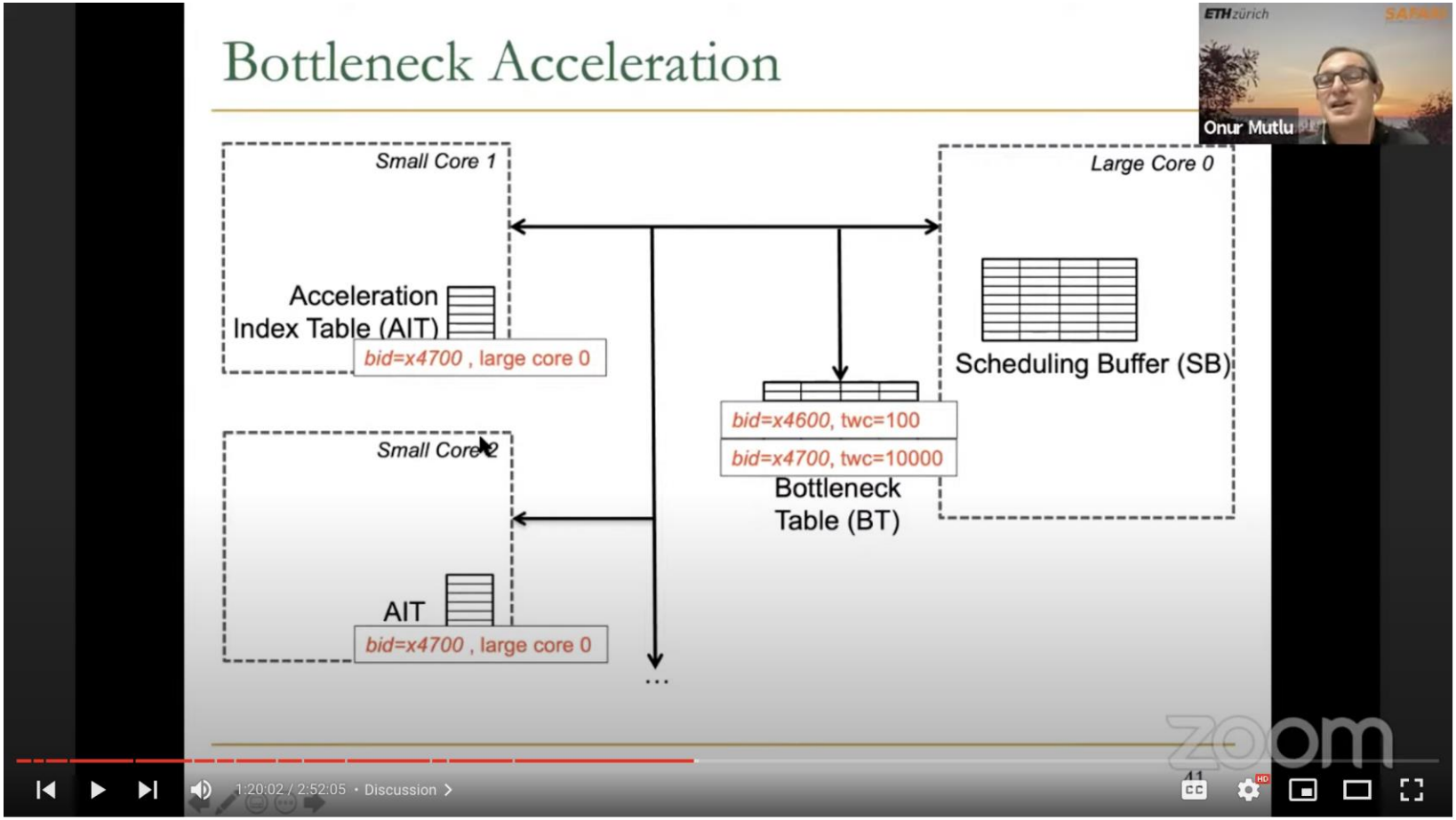
Onur Mutlu Lectures
29.2K subscribers

Analytics | Edit video | 37 | Share | Download | Clip | Save | ...

1,589 views Streamed live on Nov 25, 2021
Computer Architecture, ETH Zürich, Fall 2021 (https://safari.ethz.ch/architecture/f...)

https://www.youtube.com/watch?v=GLzG_rEDn9A&list=PL5Q2soXY2Zi-Mnk1PxjEIG32HAGILkTOF&index=19

Lecture on Bottleneck Acceleration



https://www.youtube.com/watch?v=P8l3SMAbyYw&list=PL5Q2soXY2Zi-Mnk1PxjEIG32HAGILkTOF&index=19

An Example Parallel Problem: Task Assignment to Processors

Static versus Dynamic Scheduling

- Static: Done at compile time or parallel task creation time
  - Schedule does not change based on runtime information
- Dynamic: Done at run time (e.g., after tasks are created)
  - Schedule changes based on runtime information
- Example: Instruction scheduling
  - Why would you like to do dynamic scheduling?
  - What pieces of information are not available to the static scheduler (i.e., compiler)?
    - Branch direction, memory address, (memory) operation latency

Parallel Task Assignment: Tradeoffs

- Problem: N tasks, P processors, N>P. Do we assign tasks to processors statically (fixed) or dynamically (adaptive)?

- Static assignment
  - + Simpler: No movement of tasks
  - - Inefficient: Underutilizes resources when load is not balanced

    When can load not be balanced?

- Dynamic assignment
  - + Efficient: Better utilizes processors when load is not balanced
  - - More complex: Need to move tasks to balance processor load
  - - Higher overhead: Task movement takes time, can disrupt locality

Parallel Task Assignment: Example

- Compute histogram of a large set of values
- Parallelization:
  - Divide the values across T tasks
  - Each task computes a local histogram for its value set
  - Local histograms merged with global histograms in the end

```
GetPageHistogram(Page *P)

    For each thread: {

        /* Parallel part of the function */
        UpdateLocalHistogram(Fraction of Page)

        /* Serial part of the function */
        Critical Section:
            Add local histogram to global histogram

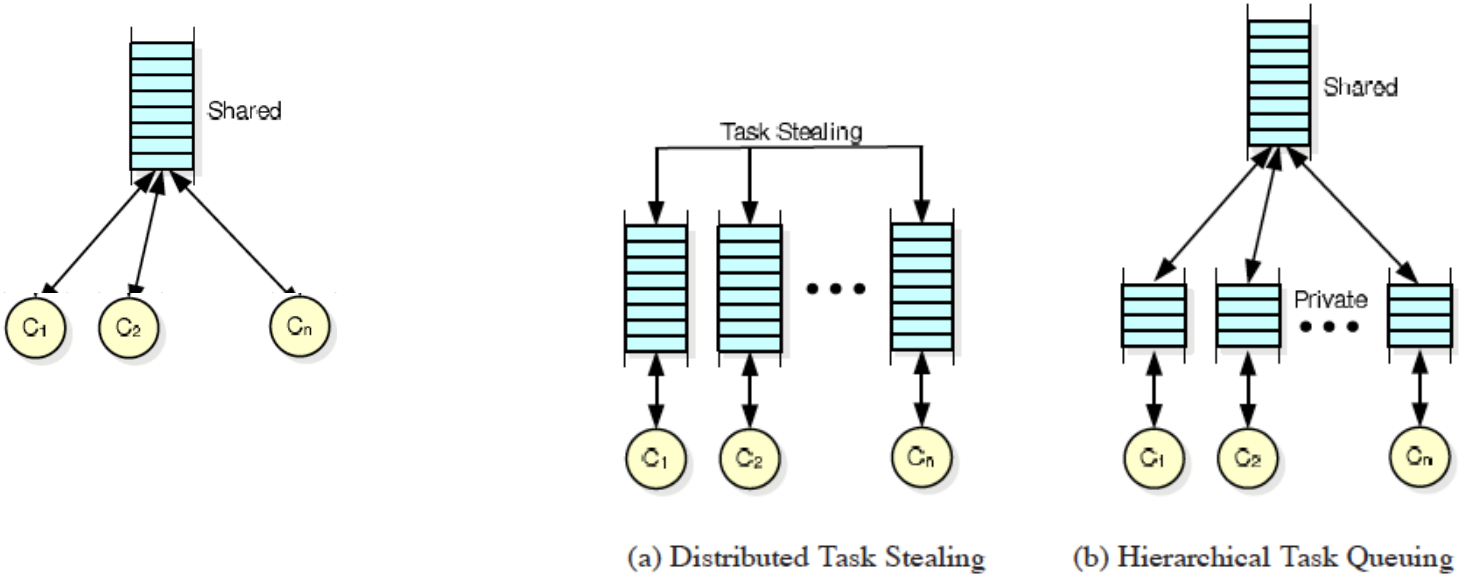
        Barrier
    }

    Return global histogram
```

Parallel Task Assignment: Example (II)

- How to schedule tasks updating local histograms?
  - Static: Assign equal number of tasks to each processor
  - Dynamic: Assign tasks to a processor that is available
  - When does static work as well as (or better than) dynamic?

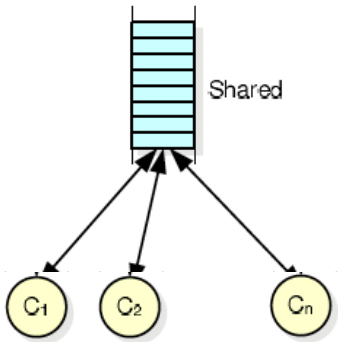
- Implementation of Dynamic Assignment with Task Queues

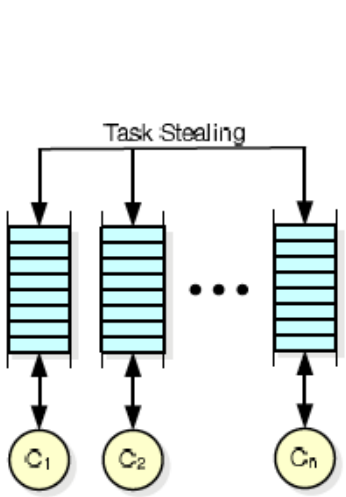


(a) Distributed Task Stealing (b) Hierarchical Task Queuing

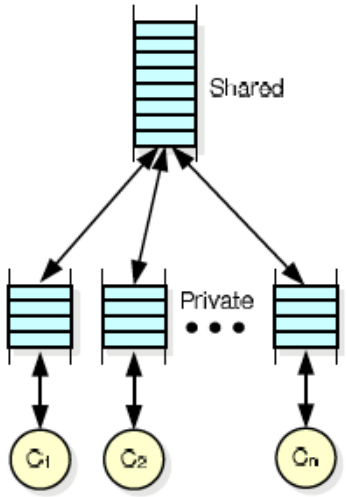
Software Task Queues

- What are the advantages and disadvantages of each?
  - Centralized
  - Distributed
  - Hierarchical





(a) Distributed Task Stealing



(b) Hierarchical Task Queuing

Task Stealing

- Idea: When a processor's task queue is empty it steals a task from another processor's task queue
  - Whom to steal from? (Randomized stealing works well)
  - How many tasks to steal?

+ Dynamic balancing of computation load

- Additional communication/synchronization overhead between processors
- Need to stop stealing if there are no tasks to steal

Parallel Task Assignment: Tradeoffs

- Who does the assignment? Hardware versus software?

- Software
  - + Better scope
  - - More time overhead
  - - Slow to adapt to dynamic events (e.g., a processor becoming idle)

- Hardware
  - + Low time overhead
  - + Can adjust to dynamic events faster
  - - Requires hardware changes (area and possibly energy overhead)

How Can the Hardware Help?

- Managing task queues in software has overhead
  - Especially high when task sizes are small

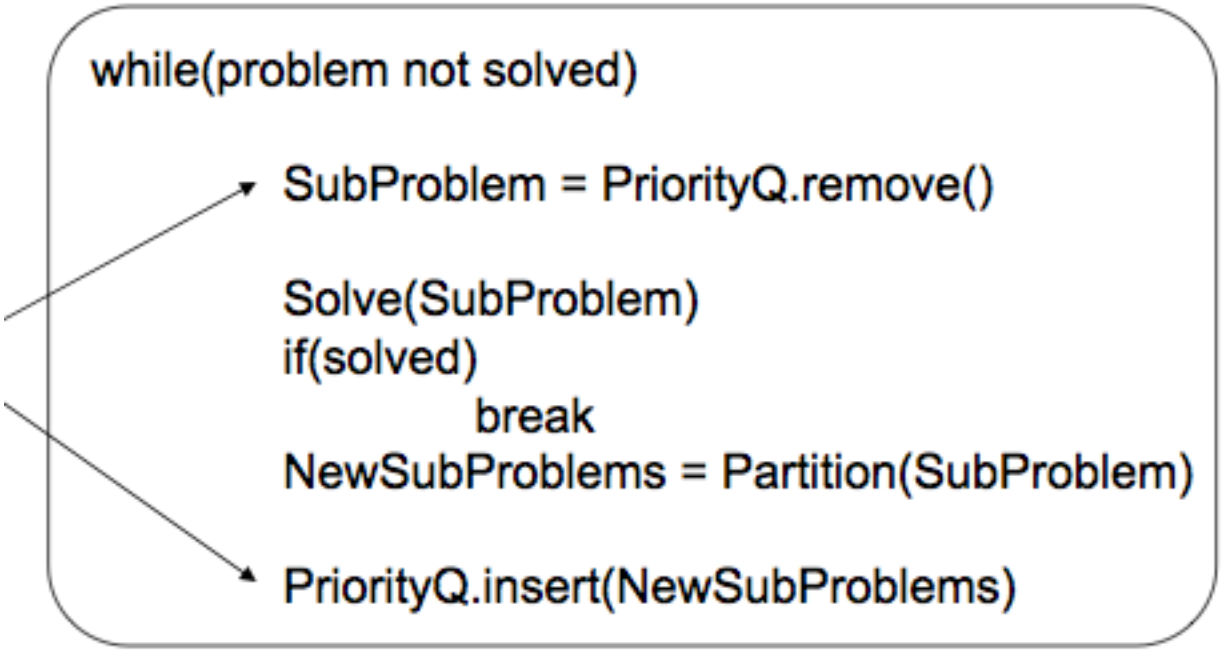
- An idea: Hardware Task Queues
  - Each processor has a dedicated task queue
  - Software fills the task queues (on demand)
  - Hardware manages movement of tasks from queue to queue
  - There can be a global task queue as well → hierarchical tasking in hardware

  - Kumar et al., "Carbon: Architectural Support for Fine-Grained Parallelism on Chip Multiprocessors," ISCA 2007.
    - Optional reading

Dynamic Task Generation

- Does static task assignment work in all cases/problems?

- What if tasks are generated dynamically
  - E.g., problem: Searching the exit of a maze



Computer Architecture

Lecture 20b: Multiprocessors

Prof. Onur Mutlu

ETH Zürich

Fall 2025

28 November 2025

Programming Model vs. Hardware Execution Model

Programming Models vs. Architectures

- Five major models
  - (Sequential)
  - Shared memory
  - Message passing
  - Data parallel (SIMD)
  - Dataflow
  - Systolic

- Hybrid models?

Shared Memory vs. Message Passing

- Are these programming models or execution models supported by the hardware architecture?
- Does a multiprocessor that is programmed by "shared memory programming model" have to support a shared address space processors?
- Does a multiprocessor that is programmed by "message passing programming model" have to have no shared address space between processors?

Programming Models: Message Passing vs. Shared Memory

- Difference: how communication is achieved between tasks
- Message passing programming model
  - Explicit communication via messages
  - Loose coupling of program components
  - Analogy: telephone call or letter, no shared location accessible to all
- Shared memory programming model
  - Implicit communication via memory operations (load/store)
  - Tight coupling of program components
  - Analogy: bulletin board, post information at a shared space
- Suitability of the programming model depends on the problem to be solved. Issues affected by the model include:
  - Overhead, scalability, ease of programming, bugs, match to underlying hardware, …

Message Passing vs. Shared Memory Hardware

- Difference: how task communication is supported in hardware
- Shared memory hardware (or machine model)
  - All processors see a global shared address space
    - Ability to access all memory from each processor
  - A write to a location is visible to the reads of other processors
- Message passing hardware (machine model)
  - No global shared address space
  - Send and receive variants are the only method of communication between processors (much like networks of workstations today, i.e. clusters)
- Suitability of the hardware depends on the problem to be solved as well as the programming model.

Programming Model vs. Hardware

- Most of parallel computing history, there was no separation between programming model and hardware
  - Message passing: Caltech Cosmic Cube, Intel Hypercube, Intel Paragon
  - Shared memory: CMU C.mmp, Sequent Balance, SGI Origin.
  - SIMD: ILLIAC IV, CM-1

- However, any hardware can really support any programming model
- Why?
  - Application → compiler/library → OS services → hardware

4th Data Prefetching Championship



More info: https://sites.google.com/view/dpc4-2026/
Infrastructure: https://github.com/CMU-SAFARI/ChampSim

Lab 4 is based on the same infrastructure.
You have a head start! Submit your ideas!

If You Want to Do Impactful Research

Please Contact Us for Doing Research

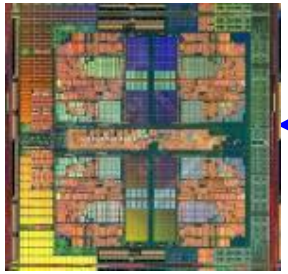
- We do research in all aspects of computer architecture, HW/SW co-design, systems, (hardware) security, bioinformatics
  - **All topics of this Comp Arch course and more!**

- We are always looking for bright and self-motivated students
  - who have a passion for rigorously solving problems
  - who are willing to do proper justice to scientific exploration, evaluation & communication

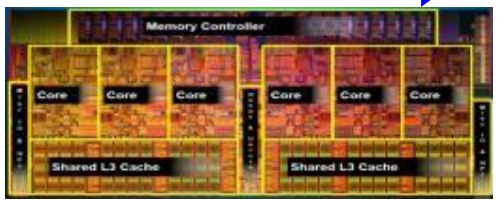
- We cherish working with **students from all over the world** with diverse backgrounds
  - in an **open and inclusive environment that fosters and values creativity, free exploration, collaboration, rigor**

SAFARI Research Group: Current Mission

Computer architecture, HW/SW, systems, bioinformatics, security



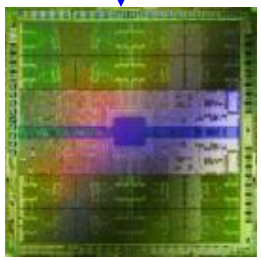
Heterogeneous Processors and Accelerators



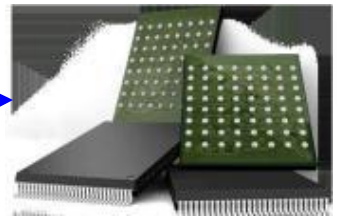


Processing in Memories





ML/AI, Vision, Genomics,
Biological Signal Processing,
Emerging Workloads and Algorithms





Storage, Sensors, Memory, Interconnect
with Processing Capabilities
+ New Paradigms for Computation
+ New Technologies

https://safari.ethz.ch

SAFARI
SAFARI Research Group
safari.ethz.ch

Enable fundamentally better computers

https://safari.ethz.ch/safari-newsletter-july-2024/

SAFARI Newsletter July 2024 Edition

- **https://safari.ethz.ch/safari-newsletter-july-2024/**



SAFARI Newsletter June 2023 Edition

- **https://safari.ethz.ch/safari-newsletter-june-2023/**

SAFARI
SAFARI Research Group

Think Big, Aim High

ETH zürich

View in your browser
June 2023



SAFARI Newsletter December 2021 Edition

- **https://safari.ethz.ch/safari-newsletter-december-2021/**

SAFARI
SAFARI Research Group

Think Big, Aim High

ETH zürich

View in your browser
December 2021



SAFARI Newsletter January 2021 Edition

- **https://safari.ethz.ch/safari-newsletter-january-2021/**

SAFARI
SAFARI Research Group

Newsletter
January 2021

Think Big, Aim High, and
Have a Wonderful 2021!



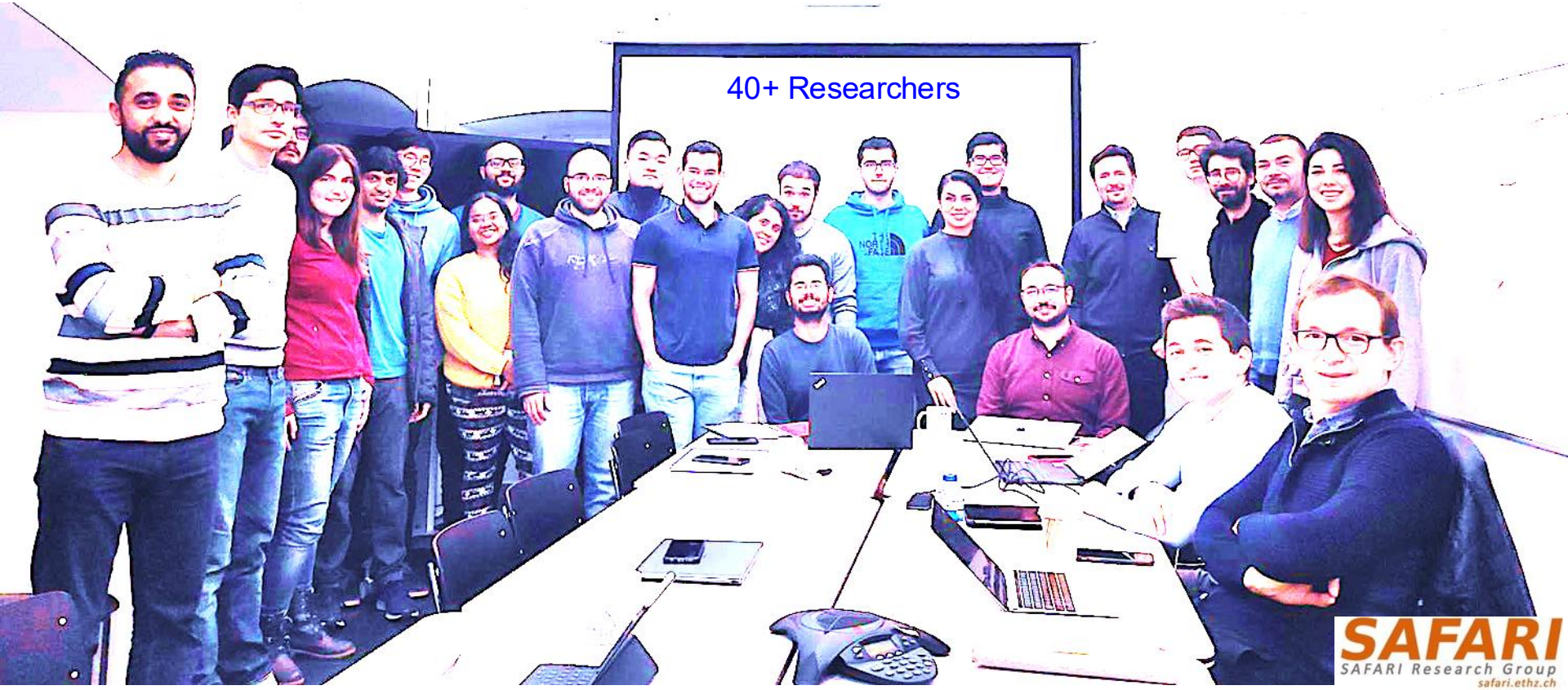
Dear SAFARI friends,

Happy New Year! We are excited to share our group highlights with you in this second edition of the SAFARI newsletter (You can find the first edition from April 2020 here). 2020 has

SAFARI Newsletter April 2020 Edition

Computer architecture, HW/SW, systems, bioinformatics, security, memory

https://safari.ethz.ch/safari-newsletter-april-2020/



Think BIG, Aim HIGH!

SAFARI

https://safari.ethz.ch

SAFARI PhD and Post-Doc Alumni

- Rahul Bera (ETH Zurich), MICRO 2022 Best Paper Award, ISCA 2024 Best Paper Award
- Geraldo Francisco de Oliveira Jr. (ETH Zurich)
- Can Firtina (UMD, Assistant Professor)
- Lukas Breitwieser (CERN), 2025 Impact Award of the Research Software Engineering Society, Best artifact award at PPoPP 2023
- A. Giray Yaglikci (CISPA), 2025 SIGMICRO Dissertation Award, DSN Carter Award Best Thesis 2025; IEEE TCCA/ACM SIGARCH Dissertation Award HM 2025, PACT 2023 SRC Winner, Intel HWSec Award Finalist 2021, HOST PhD Competition Finalist 2024, VTS PhD Competition Finalist 2025, HPCA HoF 2025, ISCA HoF 2025, Best Artifact award at HPCA 2025/ISCA 2023…
- Hasan Hassan (Rivos), EDAA Outstanding Dissertation Award 2023; S&P 2020 Best Paper Award, 2020 Pwnie Award, IEEE Micro TP HM 2020
- Christina Giannoula (MPI-SWS), NTUA Best Dissertation Award 2023
- Minesh Patel (Rutgers, Asst. Prof.), DSN Carter Award Best Thesis 2022; ETH Doctoral Medal 2023; MICRO'20 & DSN'19 Best Paper Awards; ISCA HoF 2021
- Damla Senol Cali (Bionano Genomics), SRC TECHCON 2019 Best Student Presentation Award; RECOMB-Seq 2018 Best Poster Award
- Nastaran Hajinazar (Intel)
- Gagandeep Singh (AMD/Xilinx), FPL 2020 Best Paper Award Finalist
- Amirali Boroumand (Stanford Univ → Google), SRC TECHCON 2018 Best Presentation Award
- Jeremie Kim (Apple), EDAA Outstanding Dissertation Award 2020; IEEE Micro Top Picks 2019; ISCA/MICRO HoF 2021
- Nandita Vijaykumar (Univ. of Toronto, Assistant Professor), ISCA Hall of Fame 2021
- Kevin Hsieh (Microsoft Research, Principal Researcher)
- Justin Meza (Facebook), HiPEAC 2015 Best Student Presentation Award; ICCD 2012 Best Paper Award
- Mohammed Alser (Georgia State Univ., Assistant Professor), IEEE Turkey Best PhD Thesis Award 2018
- Yixin Luo (Google → OpenAI), HPCA 2015 Best Paper Session
- Kevin Chang (Facebook), SRC TECHCON 2016 Best Student Presentation Award
- Rachata Ausavarungnirun (KMUNTB, Assistant Professor → MangoBoost), NOCS 2015 and NOCS 2012 Best Paper Award Finalist
- Gennady Pekhimenko (Univ. of Toronto, Associate Professor), ISCA Hall of Fame 2021; ASPLOS 2015 SRC Winner
- Vivek Seshadri (Microsoft Research, Principal Researcher)
- Donghyuk Lee (NVIDIA Research, Senior Researcher), HPCA Hall of Fame 2018
- Yoongu Kim (Software Robotics → Google), IFIP JCL Award'24, TCAD'19 Top Pick Award; IEEE Micro Top Picks'10; HPCA'10 Best Paper Session
- Lavanya Subramanian (Intel Labs → Facebook → OpenAI)

- Samira Khan (Univ. of Virginia, Assistant Professor → Google), HPCA 2014 Best Paper Session, DSN 2025 Test of Time Award
- Saugata Ghose (Univ. of Illinois, Assistant Professor), DFRWS-EU 2017 Best Paper Award, HPCA HoF 2024, MICRO HoF 2024
- Jawad Haj-Yahya (Huawei Research Zurich, Principal Researcher → Rivos)
- Lois Orosa (Galicia Supercomputing Center, Director)
- Jisung Park (POSTECH, Assistant Professor)
- Gagandeep Singh (AMD/Xilinx, Researcher)
- Juan Gomez-Luna (NVIDIA, Researcher), ISPASS 2023 Best Paper Session, MICRO HoF 2022
- Mohammed Alser (Georgia State Univ., Assistant Professor), IEEE Turkey Best PhD Thesis Award 2018
- Haiyu Mao (King's College, Assistant Professor)
- Yu (Lenny) Liang (INRIA Paris, Researcher)

https://safari.ethz.ch/safari-alumni/

SAFARI Introduction & Research

Computer architecture, HW/SW, systems, bioinformatics, security, memory



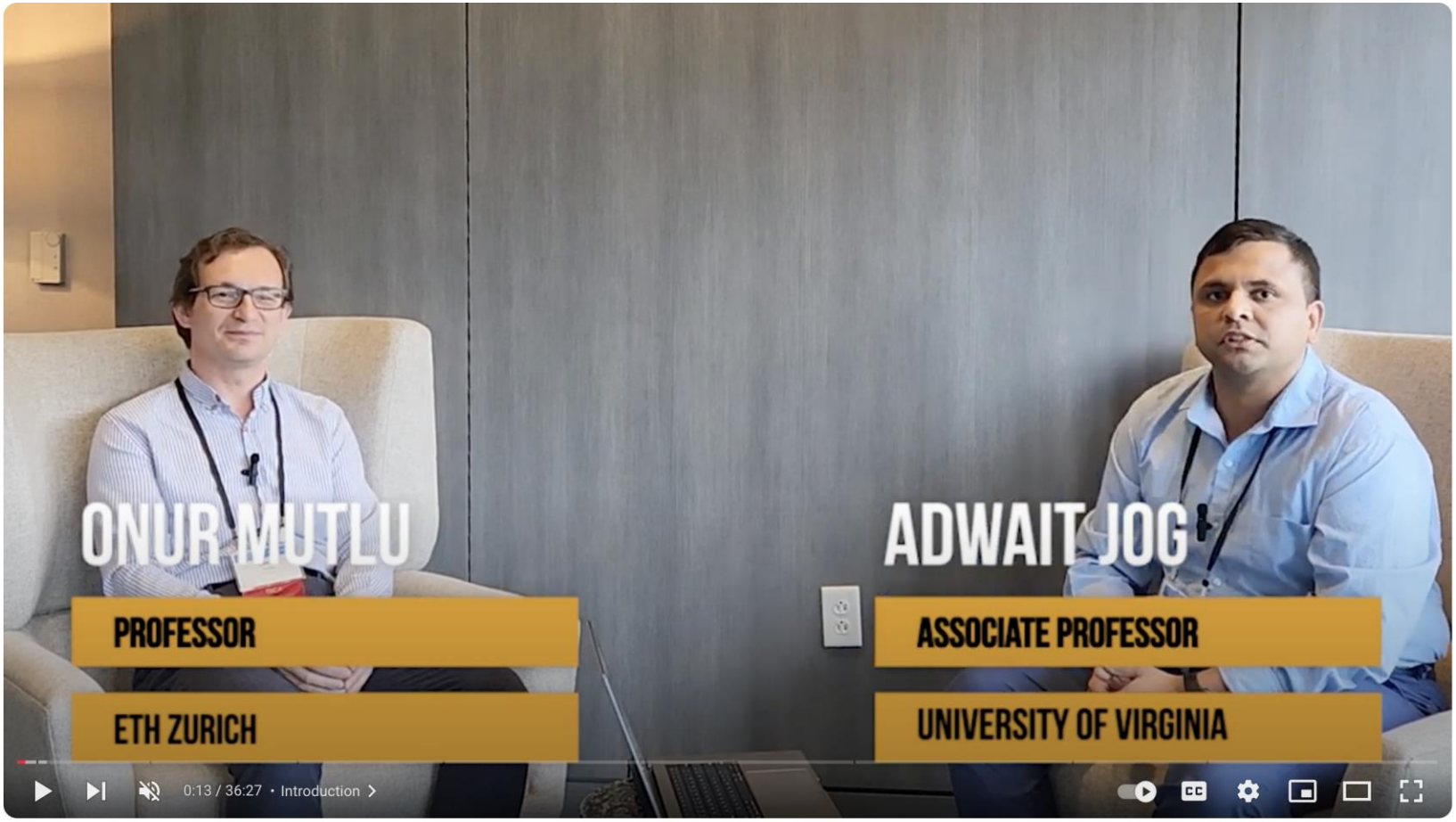
SAFARI
SAFARI Research Group
safari.ethz.ch

Think BIG, Aim HIGH!

SAFARI

https://www.youtube.com/watch?v=mV2OuB2djEs

An Interview at ISCA 2023



TCuARCH meets Prof. Onur Mutlu - 2023

Onur Mutlu Lectures
46.5K subscribers

Analytics | Edit video | 27 | Share | Promote | Download | Clip

454 views Premiered May 6, 2024

Prof. Onur Mutlu (https://people.inf.ethz.ch/omutlu/) shares his mindset seeing his life and career as a researcher, mentor, and advisor. His last remark should resonate with many young researchers. "Do what you are really excited about. Everything will follow."

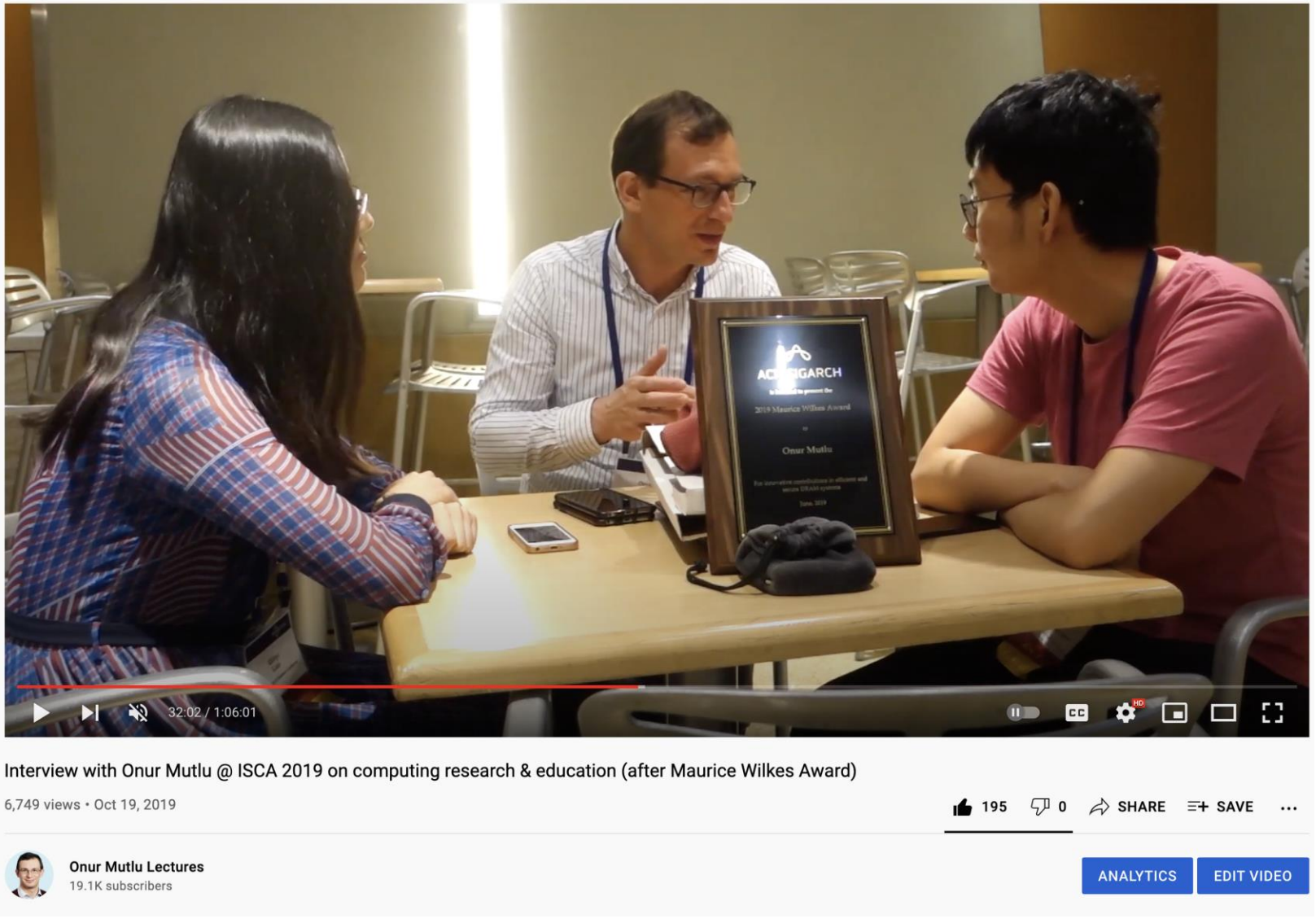
https://www.youtube.com/watch?v=6Hpn4SAX0dI

Suggestions on Research, Education, PhD



https://www.youtube.com/watch?v=83tlorht7Mc&list=PL5Q2soXY2Zi8D_5MGV6EnXEJHnV2YFBJl&index=54

A Longer Interview on Computing Futures



https://www.youtube.com/watch?v=8ffSEKZhmvo

For More Information: SAFARI Website



Think BIG, Aim HIGH!

https://safari.ethz.ch

https://people.inf.ethz.ch/omutlu/projects.htm

Research Opportunities

- If you are interested in doing research in Computer Architecture, HW/SW, Security, Systems & Bioinformatics:
  - Email me with your interest (CC: Mohammad)
  - Take our courses (online or in-person)
  - Do readings and critical reviews & **talk with us**

- There are many exciting projects and research topics, e.g.:
  - Novel memory/storage/computation/communication systems
  - New execution paradigms (e.g., in-memory computing)
  - Hardware security, safety, reliability, predictability
  - GPUs, TPUs, FPGAs, PIM, heterogeneous systems, …
  - Security-architecture-reliability-energy-performance interactions
  - Architectures for genomics/proteomics/medical/health/AI/ML
  - A very limited list is here: https://safari.ethz.ch/theses/

https://people.inf.ethz.ch/omutlu/projects.htm

You Can Join Us!

- **https://safari.ethz.ch/apply/**

SAFARI Researcher Applications

Sign in

This is the application submission site to be considered for being a researcher in the SAFARI Research Group, directed by Professor Onur Mutlu (Publications and Teaching).

If you are interested in doing research in the SAFARI Research Group, please make sure you apply through this submissions site and supply as many of the requested documents and information as possible. Please read and follow the provided instructions and submit as complete an application as possible (given the position you are applying for).

We suggest studying the following materials before submission:
SAFARI Publications and Courses
Onur Mutlu's Online Lectures and Course Materials

We strongly recommend that you read and analyze critically as many recent papers from our group as possible. This is the best way to prepare for the application process. Our recommendation is that you use professor Mutlu's methodology for critically analyzing papers.
Guide On Reviewing Papers

Good luck!

Welcome to the SAFARI at ETH Zurich -- PhD, Postdoc, Internship, Visiting Researcher Applications (SAFARI Researcher Applications) submissions site.